



# Detecting Self-Implemented Cryptographic Functions in Codebases

## BACHELOR THESIS

As part of the examination:

**Bachelor of Science (B. Sc.)**

in Computer Science

at the

Baden-Württemberg Cooperative State University Karlsruhe

by

**Florian Zürner**

**Submission Date** 31. August 2026

**Processing Period** 12 Weeks

**Matriculation Number, Course** 7916970, TINF23B2

**Training Company** SAP SE

Dietmar-Hopp-Allee 16

69190 Walldorf, Deutschland

**Supervisor of the Training Company** Nikolas Krätzschar

**Reviewer of the Duale Hochschule** Dr. Arne Babenhauserheide

# Declaration of Originality

I hereby acknowledge, that my Bachelor Thesis with the topic:

*Detecting Self-Implemented Cryptographic Functions in Codebases*

according to §4 of the “Satzung über die studienbereichsspezifischen Regelungen für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden-Württemberg” from 07.03.2024 has been independently written and no other sources or aids other than those specified have been used.

Furthermore, I confirm that the submitted electronic version corresponds to the printed version.

Karlsruhe, 30. August 2026

---

Zürner, Florian

## **ABSTRACT**

*- Deutsch -*

Diese Arbeit entwickelt einen automatisierten Workflow zur Identifikation selbstimplementierter kryptographischer Funktionen mit dem Ziel, Garden Linux beim Erreichen der FIPS 140-3 Konformität zu unterstützen. Der vorgeschlagene Ansatz erstellt hochdimensionale Vektor-Embeddings aus Code-Dateien und klassifiziert diese mittels binärer Klassifikatoren. Die robusteste Konfiguration erzielt einen Recall von 93%. Eine Fallstudie auf der systemd Codebase zeigt die Existenz von API freien SHA-256-Implementierungen sowie weiterer Kandidaten, wie z. B. SipHash.

## **ABSTRACT**

*- English -*

This work develops an automated workflow for identifying self-implemented cryptographic functions, with the aim of supporting Garden Linux in achieving FIPS 140-3 compliance. The proposed approach generates high-dimensional vector embeddings from code files and classifies them using binary classifiers. The most robust configuration achieves a Recall score of 93%. A case study on the open source project “systemd” demonstrates the presence of API-free SHA-256 implementations as well as other candidates, such as SipHash, within its codebase.

# Table of Contents

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| List of Abbreviations .....                                                        | VI        |
| List of Figures .....                                                              | VII       |
| Table Directory .....                                                              | VIII      |
| <b>1 Introduction .....</b>                                                        | <b>1</b>  |
| <b>2 Theoretical Background .....</b>                                              | <b>3</b>  |
| 2.1 FIPS compliance .....                                                          | 3         |
| 2.2 Classification Methods .....                                                   | 3         |
| 2.2.1 Random Forest Classification .....                                           | 4         |
| 2.2.2 Neural Network Classification .....                                          | 5         |
| 2.3 Classification Metrics .....                                                   | 7         |
| 2.4 Model Training .....                                                           | 9         |
| 2.4.1 Training and Validation Methodologies .....                                  | 10        |
| 2.4.2 Model Evaluation .....                                                       | 10        |
| 2.4.3 Model Comparison .....                                                       | 12        |
| 2.5 Large Language Models .....                                                    | 13        |
| 2.5.1 History of Natural Language Processing .....                                 | 13        |
| 2.5.2 Tokenizing .....                                                             | 14        |
| 2.5.3 Vector Embeddings .....                                                      | 14        |
| 2.6 Fine-tuning a Large Language Model .....                                       | 16        |
| 2.6.1 Full Fine-tuning .....                                                       | 16        |
| 2.6.2 Partial Fine-tuning .....                                                    | 16        |
| 2.6.3 Parameter-Efficient Fine-Tuning .....                                        | 17        |
| <b>3 Context .....</b>                                                             | <b>18</b> |
| 3.1 Garden Linux .....                                                             | 18        |
| 3.2 FIPS Relevance for Garden Linux .....                                          | 18        |
| 3.3 Related Work .....                                                             | 19        |
| 3.3.1 CryptoGo .....                                                               | 19        |
| 3.3.2 CryptoGuard .....                                                            | 19        |
| 3.3.3 Detection of cryptographic algorithms with grap .....                        | 20        |
| 3.3.4 Aligot .....                                                                 | 21        |
| 3.3.5 Identification of cryptographic algorithms in binary programs .....          | 22        |
| 3.3.6 CryptoLLM .....                                                              | 22        |
| 3.3.7 CryptoScope .....                                                            | 23        |
| 3.3.8 FoC .....                                                                    | 24        |
| 3.3.9 Automated Recognition of Cryptographic Algorithms via Machine Learning ..... | 25        |
| 3.3.10 Pattern Recognition using LLMs .....                                        | 25        |
| 3.3.11 Comparison of Related Work .....                                            | 25        |
| <b>4 Methodology .....</b>                                                         | <b>28</b> |
| 4.1 Dataset Construction .....                                                     | 28        |

|          |                                                                         |            |
|----------|-------------------------------------------------------------------------|------------|
| 4.1.1    | Dataset Requirements .....                                              | 28         |
| 4.1.2    | Data Gathering and Filtering .....                                      | 29         |
| 4.1.3    | Labeling .....                                                          | 29         |
| 4.1.4    | Dataset Limitations .....                                               | 31         |
| 4.2      | Evaluation Metrics .....                                                | 32         |
| 4.3      | Model Pipeline .....                                                    | 33         |
| 4.4      | Hyperparameter Tuning .....                                             | 34         |
| 4.4.1    | Embedding and Classifier parameters .....                               | 35         |
| 4.4.2    | Running the Tests .....                                                 | 40         |
| 4.4.3    | Summary .....                                                           | 46         |
| 4.5      | Building a Detection Model Based on a Fine-Tuned Large Language Model . | 47         |
| 4.5.1    | Identifying Cryptographic Chunks for Fine-Tuning .....                  | 48         |
| 4.5.2    | Pipeline of the Extended Detection Model .....                          | 49         |
| 4.5.3    | Evaluating the Fine-Tuned Approach .....                                | 50         |
| <b>5</b> | <b>Evaluation .....</b>                                                 | <b>53</b>  |
| 5.1      | Model Evaluation .....                                                  | 53         |
| 5.2      | Model Explainability .....                                              | 54         |
| 5.3      | Detecting Self-Implemented Cryptographic Code in Systemd .....          | 56         |
| 5.4      | Comparison with Related Work .....                                      | 58         |
| 5.5      | Economic Aspects .....                                                  | 60         |
| 5.6      | Sustainability .....                                                    | 61         |
| <b>6</b> | <b>Conclusion .....</b>                                                 | <b>64</b>  |
| 6.1      | Summary of Findings .....                                               | 64         |
| 6.2      | Contributions .....                                                     | 65         |
| 6.3      | Practicalities for Garden Linux and FIPS Certification .....            | 65         |
| 6.4      | Limitations .....                                                       | 66         |
| 6.5      | Future Work .....                                                       | 67         |
| 6.6      | Final Remarks .....                                                     | 68         |
|          | <b>Bibliography .....</b>                                               | <b>IX</b>  |
|          | <b>Appendices .....</b>                                                 | <b>XIV</b> |

## List of Abbreviations

|             |                                         |
|-------------|-----------------------------------------|
| <i>AST</i>  | Abstract Syntax Tree                    |
| <i>AUC</i>  | Area Under the ROC Curve                |
| <i>BCE</i>  | Binary Cross Entropy                    |
| <i>CFG</i>  | Control Flow Graph                      |
| <i>CMVP</i> | Cryptographic Module Validation Program |
| <i>CNN</i>  | Convolutional Neural Network            |
| <i>CoT</i>  | Chain-of-Thought                        |
| <i>DFG</i>  | Data Flow Graph                         |
| <i>FIPS</i> | Federal Information Processing Standard |
| <i>GCN</i>  | Graph Convolutional Network             |
| <i>GELU</i> | Gaussian Error Linear Unit              |
| <i>LLM</i>  | Large Language Model                    |
| <i>NLP</i>  | Natural Language Processing             |
| <i>OOBs</i> | Out Of Bag samples                      |
| <i>PEFT</i> | Parameter-Efficient Fine-Tuning         |
| <i>RAG</i>  | Retrieval-Augmented Generation          |
| <i>ROC</i>  | receiver operating characteristic       |
| <i>SIMD</i> | Single Instruction Multiple Data        |
| <i>VRAM</i> | Visual Random Access Memory             |

## List of Figures

|           |                                                                               |    |
|-----------|-------------------------------------------------------------------------------|----|
| Figure 1  | Visualization of Random Forest training and prediction (Sec. 2.2.1) . . . . . | 5  |
| Figure 2  | Neural network binary classifier implementation (Sec. 2.2.2) . . . . .        | 7  |
| Figure 3  | Relationships of embedding vectors (Sec. 2.5.3) . . . . .                     | 15 |
| Figure 4  | Tabular comparison of related work (Sec. 3.3.11) . . . . .                    | 25 |
| Figure 5  | Dataset Splitting Visualization (Sec. 4.1.3) . . . . .                        | 30 |
| Figure 6  | Pipeline of the detection model's training process (Sec. 4.3) . . . . .       | 33 |
| Figure 7  | 3 Layer Classifier Architecture (Sec. 4.4.1) . . . . .                        | 39 |
| Figure 8  | All F1-Scores for 4860 classifiers (Sec. 4.4.2) . . . . .                     | 41 |
| Figure 9  | All F1-Scores for best 50 classifiers (Sec. 4.4.2) . . . . .                  | 44 |
| Figure 10 | Best classifier decision thresholds (Sec. 4.4.2) . . . . .                    | 45 |
| Figure 11 | Improved detection model's training process (Sec. 4.5.2) . . . . .            | 49 |
| Figure 12 | Extended model compared with basic model. (Sec. 4.5.3) . . . . .              | 51 |
| Figure 13 | Classification Results of systemd (Sec. 5.3) . . . . .                        | 56 |
| Figure 14 | GPU Power consumption (Sec. 5.6) . . . . .                                    | 62 |



## Table Directory

|          |                                                                            |    |
|----------|----------------------------------------------------------------------------|----|
| Table 1  | Confusion matrix and binary outcomes for classification (Sec. 2.3) .....   | 8  |
| Table 2  | Confusion matrix and example for model comparison (Sec. 2.4.3) .....       | 12 |
| Table 3  | Comparison of evaluated language models (Sec. 4.4.1) .....                 | 36 |
| Table 4  | List of Chunk Overlap Configurations tested (Sec. 4.4.1) .....             | 36 |
| Table 5  | Tested hyperparameters for the Random Forest classifier (Sec. 4.4.1) ....  | 37 |
| Table 6  | Tested hyperparameters for the neural network classifier (Sec. 4.4.1) .... | 38 |
| Table 7  | Thresholds optimized for different metrics (Sec. 4.4.3) .....              | 46 |
| Table 8  | Classifier scores on test set (Sec. 5.1) .....                             | 53 |
| Table 9  | Detection results at the selected Recall threshold. (Sec. 5.1) .....       | 54 |
| Table 10 | Classification Results for Permutations (Sec. 5.2) .....                   | 55 |
| Table 11 | Average token usages of Claude and Gemini (Sec. 5.5) .....                 | 61 |

# 1 Introduction

The open source community consists of dedicated people, often volunteers, who maintain and publish applications that build the foundation of the internet as we know it today [1]. According to the latest Open Source Risk Analysis Report by BlackDuck, 98% of commercial codebases contain open source software [2]. These software projects typically depend on large numbers of external components, making it difficult to keep all dependencies up to date. Sonatype reports that a typical application contains around 180 open source components and that 80% of application dependencies remained un-upgraded for more than one year in its 2024 analysis [3].

Among this large amount of code, it is inevitable that vulnerabilities occur, which might especially happen when open source maintainers must contribute in their limited free time [1]. The same BlackDuck study shows that 87% of the evaluated codebases contain at least one vulnerability. Additionally, the challenging task of maintaining open source projects can lead to poor design choices, where code components are re-implemented instead of using standard and reviewed packages. The BlackDuck report found that 16% of open source components use non-standard code, although a standardized package exists. This poses an issue for risk analysis because tools that analyze code for vulnerabilities and misuses might miss such code [2].

One important component of software, where misuse can affect security, are cryptographic modules. Cryptographic implementations are responsible for protecting the confidentiality and integrity of sensitive information. Implementing cryptography with vulnerabilities puts application security and data safety at risk [4]. Especially for governments, the protection of sensitive data has high priority. Hence, projects that aim to be usable for federal organizations in the United States are required to obtain a security certification, called Federal Information Processing Standard (FIPS) [4].

On the path to achieving FIPS 140.3 compliance, projects need to analyze and certify any use of not already certified cryptographic functions [4]. Code audits have shown that some of the open source dependencies used fall under the 16% category mentioned above. For example, systemd ships with a copy of the glibc SHA256 code and their own HMAC-SHA256 implementation, arguing that this is necessary due to license incompatibilities and the inability to link glibc early in the boot process [5]. Even if glibc as a library would have already been reviewed and certified, the standalone file would require the same reviewing process [4].

This puts redundant workload on the developers, which is why such cases need to be identified and ideally replaced with standardized APIs. Therefore, this thesis aims to develop an analysis tool to support projects seeking FIPS compliance by locating self-implemented cryptographic functions within their dependencies. The code for reproducing the training and usage of this model can be found on the following GitHub page: <https://github.com/Sagichnicht04/CryptoVectorClassification>

## 2 Theoretical Background

Before diving into the context of Garden Linux and the detection methodology developed in this thesis, it is essential to introduce the theoretical foundations. This chapter first discusses FIPS and its certification requirements. It then introduces the machine learning algorithms used for binary classification along with their evaluation metrics and training methodologies. Finally, the chapter covers natural language processing using Large Language Models (LLMs) and techniques for parameter-efficient fine-tuning.

### 2.1 FIPS compliance

The FIPS Publication 140-3 is a U.S. government computer security standard used to approve cryptographic modules. It provides a standard for secure designs, implementations, and secure operations for cryptographic uses. This standard is required for all federal agencies that use security systems containing cryptography in order to protect sensitive information [4].

The standard aims to be adaptable to future changes in science and technology in cryptography related fields. It defines four increasing levels of security, starting at level 1 and going up to level 4. These levels address different degrees of data sensitivity and are designed to protect against several types of attackers, primarily by enforcing a higher cost of successful attacks than the attacks' possible payoff [4].

To ensure the validation of the whole software, the Cryptographic Module Validation Program (CMVP) ensures testing through independent, accredited laboratories. Compliance with FIPS 140-3 requires modules to use only approved security functions, including any specific cryptographic algorithms, key management techniques, and authentication methods [4].

### 2.2 Classification Methods

To achieve FIPS compliance, all cryptographic functions within codebases must be identified. Doing so manually across large software ecosystems can take a lot of time. To automate this task, machine learning classification methods can be employed to systematically identify all relevant code files. This thesis evaluates two distinct classification methods to analyze source code. The following sections introduces these two classifiers.

### 2.2.1 Random Forest Classification

A Random Forest is an ensemble learning method that combines the predictions of multiple decision trees. The construction and prediction process of a Random Forest consists of three main steps, visualized in Figure 1: [6].

- **Bootstrapping (Bagging):** Before constructing each decision tree, a bootstrap sample is generated for that tree by randomly selecting training examples from the original dataset with replacement. With replacement means some examples may be drawn multiple times for a single tree, while others may never be used across any tree. Each tree is therefore trained on a different subset of the training data, which increases the diversity of the trees and reduces the correlation between their predictions [6]. The samples that are left out (Out Of Bag samples (OOBs)) can later be used to evaluate the model. However, the OOBs do not necessarily represent the classification classes evenly [6]. This introduces an issue for evaluation, which is further elaborated in Section 2.4.1.
- **Feature Randomization:** During tree construction, at each node, the algorithm randomly selects a subset of the available features and determines the best split among these candidate features. Typically, the best split point is determined by selecting the split with the lowest Gini impurity [7]. The selected split partitions the samples reaching the current node into child nodes. Each child node is recursively split until a stopping criterion is met, such as all samples in the node belonging to the same class (pure node) or no further split yielding an improvement in the splitting criterion [6].
- **Ensemble Aggregation:** After all decision trees have been trained, each tree independently predicts the class of a new input sample. The Random Forest combines these predictions using a voting system for classification tasks to produce the final prediction [6].

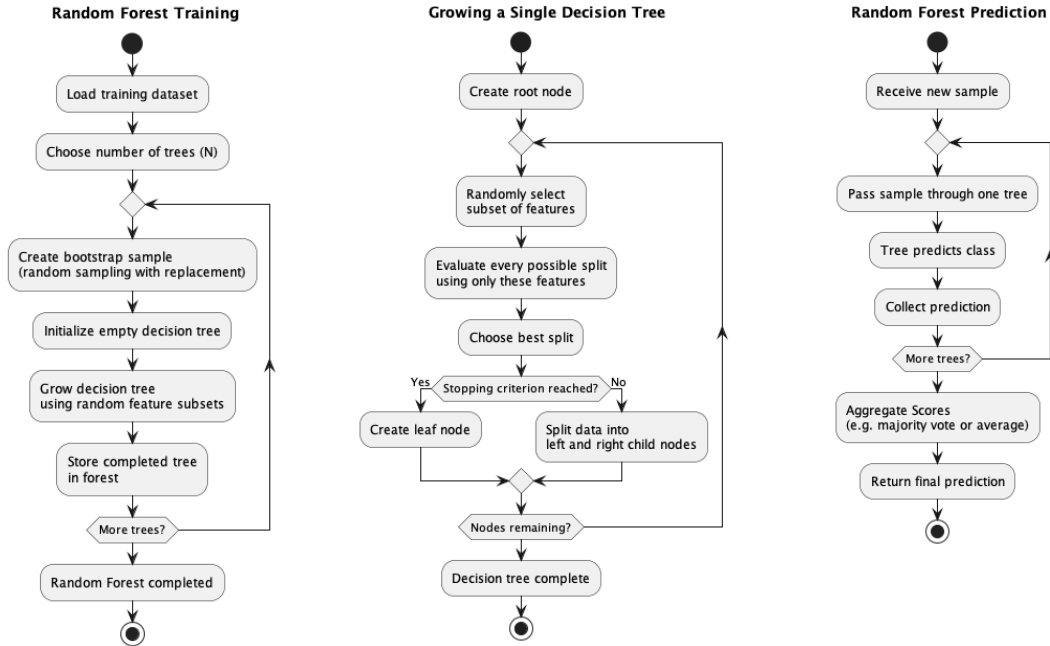


Figure 1: Visualization of Random Forest training and prediction

An important property of Random Forests is that the generalization error converges as the number of trees in the forest increases. Consequently, adding more trees typically improves the predictive performance rather than causing overfitting. Breiman, the original author of the Random Forest algorithm, proved that the generalization error converges to a limiting value as the number of trees increases. This behavior can be explained by the law of large numbers, as the predictions of an increasingly large ensemble become more stable. [6].

In addition to the forest size, implementations of the Random Forest classifier, like the Random Forest implementation by scikit-learn for Python, include additional hyperparameters that can be used to fine-tune the model's performance. For example, a maximum tree length can be set to prevent trees from getting too long and increasing computational cost. Also, a minimum number of samples required to split a tree node can be defined. Furthermore, the bootstrapping method can be disabled so that the random forest is trained on every data point. While this prevents OOBs, the introduced correlation usually reduces model performance [6, 7].

### 2.2.2 Neural Network Classification

A neural network can be understood as a complex decision-making machine, organized into layers of interconnected nodes. For a task like classifying binary classes, the network takes an input, such as a vector embedding representing a piece of source code, and

processes it through these layers. Each node in a layer receives signals from nodes in the previous layer, weighs them according to its internal parameters, and then uses an activation function to compute an output signal, which it passes on to the next layer. This process continues until the final layer, which produces a single raw output value, known as a “logit” [8].

This logit can be any number, but for binary classification, it needs to be interpreted as a value between 0 and 1 that represents the model’s confidence that the input belongs to the positive class (e. g., “is cryptographic”). To achieve this, the logit is passed through a sigmoid function. A large positive logit will result in a probability close to 1, while a large negative logit will yield a probability close to 0 [8].

Once the network produces a probability, it is still required to measure how wrong the prediction is compared to the actual label. This is the role of a loss function. For binary classification, the Binary Cross Entropy (BCE) loss function can be used. The core idea behind BCE is to heavily penalize predictions that are both confident and incorrect. It represents the difference between the predicted probability and the true binary label [8, p. 205f].

For a single training instance, the loss  $L$  is calculated as:

$$L = -(y \ln(p) + (1 - y) \ln(1 - p)),$$

where  $y$  is the actual label (0 or 1) and  $p$  is the predicted probability (between 0 and 1) [9]

Therefore, the BCE function comes with the following characteristics:

- If the true label is 1 (positive class) and the network predicts a probability close to 0, it is very confident but wrong. The BCE loss will be a very large number, signaling a significant error that needs correction.
- Conversely, if the true label is 0 (negative class) and the network predicts a probability close to 1, it is again confidently wrong, and the BCE loss will be very high.
- If the network’s predicted probability is close to the true label (e. g., a prediction of 0.95 for a true label of 1), the loss will be small, indicating a good prediction.

During training of a neural network, the goal is to minimize the total BCE loss across all examples in the training data. The network uses this loss value to adjust its internal parameters (weights) through backpropagation and an optimization algorithm like gradient descent. By iteratively modifying its weights in the direction that reduces the loss, the network gradually learns to make more accurate predictions [8, p. 236-246].

An example implementation of a neural network trained with a BCE function can be achieved with PyTorch. The implementation consists of several components:

- A neural network model (e. g., a single-layer linear neural network), which computes a logit from the input features [10].
- An instance of the `BCEWithLogitsLoss` class, which computes the binary cross-entropy loss directly from the model’s logits and the corresponding target labels. This loss is then used during backpropagation to compute the gradients of the model parameters [9, 10].
- An optimization algorithm, such as Adam, which updates the model parameters based on the computed gradients and a predefined learning rate [10].

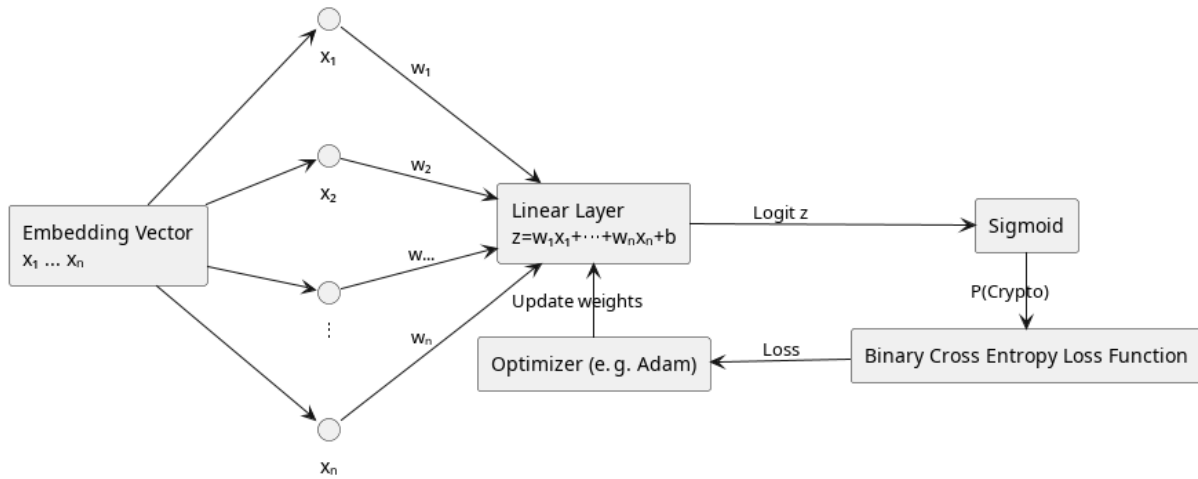


Figure 2: Visualization of an example neural network binary classifier implementation.

Apart from the learning rate, there are other hyperparameters which influence the model’s performance. For example, training for too many epochs can lead to overfitting, where the model begins to learn noise and specific characteristics which only apply to the training data, which increases the error when the model is classifying data it has not been trained on [11, p. 109f]. In addition, the batch size determines how many training samples are processed before the loss is computed and the model parameters are updated [9].

## 2.3 Classification Metrics

To evaluate the performance of a binary classifier, such as the distinction between cryptographic and non-cryptographic code, prediction metrics can be used. They can



be visualized using a confusion matrix, which categorizes the model’s predictions into four distinct outcomes [12, p. 192f]:

|                                                       | <b>Cryptographic<br/>(Actual)</b>                                               | <b>Non-Cryptographic<br/>(Actual)</b>                                           |
|-------------------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>Classified Crypto<br/>(Predicted Positive)</b>     | <b>True Positive (TP)</b><br>Crypto code correctly identified.                  | <b>False Positive (FP)</b><br>Non-crypto code incorrectly identified as crypto. |
| <b>Classified Non-Crypto<br/>(Predicted Negative)</b> | <b>False Negative (FN)</b><br>Crypto code incorrectly identified as non-crypto. | <b>True Negative (TN)</b><br>Non-crypto code correctly identified.              |

Table 1: Confusion matrix and binary outcomes for cryptographic code classification, adapted from “Introduction to Information Retrieval” [12, p. 192f].

Based on these outcomes, three performance metrics are calculated to look at different aspects of the classifier:

- **Precision ( $P$ )**: A metric that indicates how often the model is right when code is classified as crypto. It represents the ratio of true positives to all positive predictions, calculated as:

$$P = \frac{TP}{TP + FP}$$

- **Recall ( $R$ )**: A metric that indicates how often the model succeeds in classifying a crypto code snippet as such by measuring the proportion of actual crypto snippets that were correctly classified. It is calculated as:

$$R = \frac{TP}{TP + FN}$$

- **F1-Score ( $F_1$ )**: A metric that combines Precision and Recall into a single score. It provides a balanced measure of the model’s ability to correctly identify positives while limiting both false positives and false negatives. It is defined as:

$$F_1 = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

The advantage of utilizing both precision and recall is that they capture different priorities in model performance. For instance, in a security audit, high recall is important to ensure no hidden cryptographic functions are missed (minimizing false negatives),

even at the cost of tolerating some false positives. On the other side, high precision is desirable when false positives must be minimized to avoid developer fatigue. The  $F_1$ -score balances these two quantities, offering a simple comparable value [12, p. 192f].

## 2.4 Model Training

In their article “Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning”, Sebastian Raschka introduces methods on how to evaluate a model’s performance. The article acts as a comprehensive guide on how to correctly estimate a model’s generalization capabilities and compare it to other models. To achieve this, the article introduces some general concepts [13].

The author introduces “unseen data” (data the model has not been trained on) and that there is no guarantee that one’s model can predict it. When a machine learning model is trained, it is easy to assume that a high training accuracy means the model is ready for real-world tasks. However, the model might just memorize the training examples instead of learning the underlying patterns, which leads to poor performance on new, unseen data. To estimate if a model has learned anything useful, it must be evaluated on data it has never seen before during its training process [13].

Models need to be compared among other similar models to get to know how well it performs. Usually, similar models that solve the same or a similar prediction task have already been deployed somewhere. Also, a model can be trained on different architectures and hyperparameters. This creates a variety of candidate models that need to be ranked against each other to identify the most promising one. Evaluating how these models compare to one another allows to make informed decisions about which specific model should be deployed [13].

While comparing, it is important to apply the same bias to all models used. When comparing different models, a perfectly accurate, absolute performance estimate for each model is not necessary. Even if the evaluation method introduces a pessimistic bias, which means it slightly underestimates the performance of all models, this bias is acceptable as long as it affects all candidates equally. This is because the overall ranking of the models remains unchanged, which allows to still select the best performing model among them [13].

### 2.4.1 Training and Validation Methodologies

To systematically train and evaluate supervised classification models, several fundamental data splitting and partitioning strategies are employed [13]:

1. The author mentions the holdout method, where the dataset is split into training and validation data, for training and validating a method. The holdout method is the simplest approach to achieve this separation by dividing a single dataset into two parts: a training set and a test set. The training set is used to let the learning algorithm build the model, and the test set is kept strictly hidden. Once the training is complete, the model's accuracy is measured on the test set, providing an unbiased point estimate of its generalization performance [13].
2. Stratification is important to make sure training and validation data is always evenly distributed among the categories. Especially for small datasets, when a dataset is split randomly, there is a risk that one category ends up being overrepresented in the training set and underrepresented in the test set. For instance, in the task of identifying cryptographic code, a completely random split could result in a test set that contains no cryptographic snippets at all, making testing impossible. Therefore, stratification is important as it ensures that the original proportion of each category is kept identical in both the training and test subsets [14].

### 2.4.2 Model Evaluation

In order to estimate the uncertainty of a model's predictions, a confidence interval can be calculated. If it can be assumed that the predictions of a model follow a normal distribution, the confidence interval of a model can be calculated by treating each prediction on the dataset as a Bernoulli experiment:

$$f(k, n, p) = \Pr(X = k) = \binom{n}{k} p^k (1 - p)^{n-k},$$

where  $n$  is the amount of predictions made,  $k$  is the amount of correct predictions and  $p$  is the probability that a single prediction is correct [13].

Using the standard deviation, a 95% confidence interval for the number of successes can be constructed as

$$k \pm 1.96\sqrt{np(1-p)}$$

This interval estimates how many instances are expected to be classified correctly. For metrics like accuracy, which can be seen as an average of Bernoulli trials (correct or

incorrect), this interval can be adapted to a proportion. For example, the accuracy lies within the interval [13]:

$$\text{accuracy} \pm 1.96 \sqrt{\left(\frac{1}{n}\right) \text{accuracy} (1 - \text{accuracy})}$$

To make the results more representative, the model can be trained multiple times on different training datasets and to then compute an overall confidence interval. This repeated holdout validation (or **Monte Carlo Cross-Validation**) involves training the model repeatedly on newly shuffled training sets and testing it on the corresponding test sets. Later, all confidence intervals can be aggregated into a “repeated holdout estimate” which better represents the model in general [15].

However, if the experiments can be run multiple times, another method can be used which allows to construct confidence intervals for more complex metrics like the F1-Score. This metric is calculated from aggregated counts which makes its statistical distribution more complex. For this, the **bootstrap method** can be used. This resampling technique allows for the estimation of the sampling distribution of a metric without making strong assumptions about its underlying statistical properties [16]. The process for calculating a percentile-based bootstrap confidence interval is as follows [13]:

1. **Generate a Bootstrap Sample:** From the original test set of size  $n$ , a new “bootstrap sample” is created by drawing  $n$  samples with replacement.
2. **Train the model:** The model is trained based on the drawn bootstrap sample.
3. **Calculate the Metric:** The performance metric of interest (e. g., F1-score) is calculated for the model’s predictions on this bootstrap sample.
4. **Repeat:** Steps 1 and 2 are repeated to generate a distribution of the metric’s values.
5. **Compute the Confidence Interval:** The resulting distribution of metric values is used to determine the confidence interval. For a 95% confidence interval, the 2.5th and 97.5th percentiles of the distribution are typically used as the lower and upper bounds of the interval.

This bootstrap procedure provides an empirical approximation of the metric’s sampling distribution, making it a powerful tool for quantifying the uncertainty of performance estimates for complex classification metrics.

Other validation methods include:

- **K-Fold Cross-Validation:** Instead of randomly splitting the dataset into training and validation parts for every time the model is trained, the validation is split, such that

every sample is used exactly once in validation. Consequently, the amount of tests and training depends on the dataset size and how  $k$  is chosen [17, p.258ff].

- **Three-Way Holdout Method:** When tuning model hyperparameters, like the amount of nodes in a neural network layer, the dataset should be split in training, validation and test set. The model is trained using a tuned hyperparameter and evaluated on the validation set until a good value for the hyperparameter is found. Lastly, the model is tested on the test set [13].

### 2.4.3 Model Comparison

Using the evaluation numbers alone is not sufficient to clearly label a model as “good” or “bad”. These are relative positions that need to be compared to other established models, if possible. There are methods to compare the performance of different models and get statistical evidence whether one performs better than the other. For example, a  $\chi^2$ -test can be used to directly compare two models in a confusion matrix: [13]

|                    | Model 2<br>correct | Model 2<br>wrong |
|--------------------|--------------------|------------------|
| Model 1<br>correct | A                  | B                |
| Model 1<br>wrong   | C                  | D                |

|                    | Model 2<br>correct | Model 2<br>wrong |
|--------------------|--------------------|------------------|
| Model 1<br>correct | 160                | 10               |
| Model 1<br>wrong   | 5                  | 25               |

Table 2: Construction of a confusion matrix (left) and an example of a confusion matrix, resulting from a classification of 200 datasets using 'Model 1' and 'Model 2' (right).

This representation helps identifying which model performed best in the validation scenario. In the example in Table 2, a total of 200 files is used for testing. Here, “Model 2” failed to correctly classify 10 datasets which have been correctly identified by “Model 1”, while “Model 1” only failed to correctly classify 5 datasets that have been correctly identified by “Model 2”. This example suggests that “Model 1” performs better than “Model 2”. Assuming the null hypothesis that “Model 1 does not differ from Model 2”, the statistical significance of the difference between the models can be tested using the McNemar test. This test is preferred over the chi-square approximation for small test sets, where the number  $(B + C)$  is typically smaller than 50 [13, 18]. The two-sided p-value is computed as

$$p = 2 \sum_{i=B}^n \binom{n}{i} 0.5^i (1 - 0.5)^{n-i},$$

where  $n = B + C$ .

Assuming a significance level of  $\alpha = 0.05$ , the null hypothesis is rejected if the computed p-value is smaller than  $\alpha$  and otherwise cannot be rejected. For the example above, the exact McNemar test yields a p-value of approximately 0.30. Since this value is larger than 0.05, the null hypothesis cannot be rejected, indicating that there is insufficient statistical evidence to conclude that the two models differ in performance. If, instead, “Model 2” had misclassified 15 files that were correctly classified by “Model 1” (i.e.,  $B = 15$  and  $C = 5$ ), the exact McNemar test would yield a p-value of  $\approx 0.04$  which is below 0.05, leading to the rejection of the null hypothesis. In that case, “Model 1” would be considered to perform better on the validation set [13].

## 2.5 Large Language Models

In recent decades, the Natural Language Processing (NLP) field has evolved rapidly. Since computers emerged, humans have been fascinated with the thought of talking to machines the same way they talk to other humans.

### 2.5.1 History of Natural Language Processing

Context can put challenges to NLP. The understanding humans have about words with similar meaning is hard to transfer into the discrete dimension in which computers operate. While humans can create various kinds of sentences with the same meaning, computers used to have poor skills in generating sentences, especially when it comes to variety. In the early days of NLP, N-grams posed the foundation for computers to generate meaningful sentences. An N-gram is a simple language model where a computer learns how often sequences of  $n$  words occur in text. By using these statistics, it can predict the next word based on the previous  $n-1$  words. N-gram models powered many early natural language systems, but because they rely only on local word frequencies rather than understanding meaning, they often produced repetitive and nonsense responses [19, 20].

### 2.5.2 Tokenizing

A common way to encode text in NLP are tokens. A tokenizer breaks down text into smaller sequences of characters, known as subword units, often using frequency-based or probabilistic algorithms. These units are then mapped to unique numerical IDs. For example, using the SentencePiece tokenizer, the text “Hello world.” might be segmented into `He`, `ll`, `o`, `world`, `.` and then mapped to the ID sequence `151 88 21 887 6`. While a specific word or char sequence does not always encode the same way, tokenizers generally ensure that the same whole input text always produces the same tokenized output [21].

### 2.5.3 Vector Embeddings

As an alternative to traditional n-gram approaches, a new technology emerged: Vector embeddings can represent tokenized sentences, subwords or symbols as high-dimensional vectors. In their paper “Efficient Estimation of Word Representations in Vector Space,” [19] Tomas Mikolov et al. introduced the Word2Vec model to overcome the limitations of older N-gram approaches. Word2Vec learns word representations by training a neural network to recognize which words tend to appear together in similar contexts. This allows the model to encode the semantic environment of each symbol into a vector, ensuring that symbols appearing in similar contexts are mapped to nearby points in the high-dimensional space. This results in complex relationships, where algebraic vector operations can represent semantics known to humans, like visualized in Figure 3.

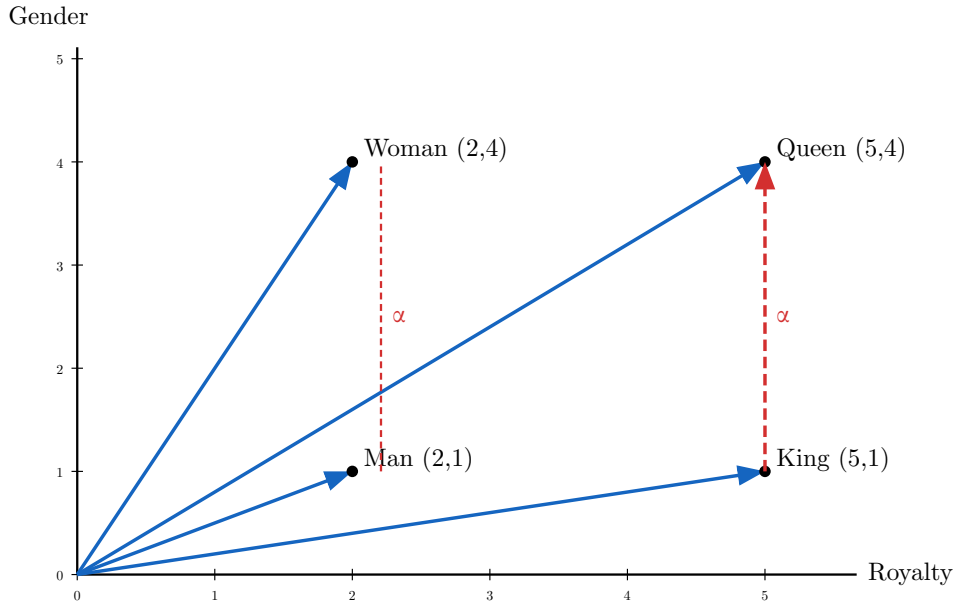


Figure 3: Visualization of relationships of embeddings in an LLM. The relationship between 'vector'(King) and 'vector'(Queen) is similar to the relationship between 'vector'(Woman) and 'vector'(Man).

For source code, specialized LLMs like CodeBERT by Feng et al. [22] adapt this concept for programming languages by training the model not only on natural language but also on large datasets of source code. By encoding code as vector representations, ideally, both syntax and semantics are captured. This allows for the comparison and classification of code snippets based on their functional meaning rather than just using keyword matches [22].

Generating text embeddings using LLMs is a computationally heavy task which requires a forward pass through multiple neural network layers. During inference, the majority of the computation consists of large matrix multiplication operations, together with attention mechanisms and other neural network operations. Since GPUs are highly optimized for massively parallel linear algebra computations, they generally provide significantly better performance for such use cases than CPUs [23].

To efficiently process text with LLMs on GPUs, most or all model parameters are typically stored in the GPU's Visual Random Access Memory (VRAM). However, the memory capacity of consumer GPUs is often limited and insufficient for loading larger LLMs. Model quantization addresses this limitation by representing model weights using lower-precision numerical formats, such as INT8 instead of FP32. While this reduces



memory consumption, the reduced numerical precision may lead to a small decrease in model accuracy [23].

## 2.6 Fine-tuning a Large Language Model

Many LLMs come with general strengths that make them distinguishable from other models. For example, some may be strong in understanding a specific language while some may be strong for coding purposes. For more specific use cases, like detecting cryptographic functions in source code, existing models can be fine-tuned to make them better suited for specific tasks [24].

Fine-tuning is the process of adapting an existing model by updating some or all of its neural network parameters on additional training data. Fine-tuning approaches can be broadly categorized into three groups:

- Full fine-tuning, where all model parameters are updated [25].
- Partial fine-tuning, where only selected layers are trained while the remaining parameters remain frozen [26].
- Parameter-efficient fine-tuning (PEFT), where only a small number of additional parameters (e. g., LoRA adapters) are optimized, substantially reducing computational and memory requirements [24].

### 2.6.1 Full Fine-tuning

In a full fine-tuning approach, all parameters of a pre-trained model are updated during training on a new, task-specific dataset. The process starts by initializing the model with the weights from the pre-training phase. During training, the entire model is updated end-to-end, allowing it to adapt its broadly learned knowledge to the specific patterns of the new task. This method is powerful and shows high performance, but it is also a very computationally expensive task. It requires significant memory to store the model's weights, gradients, and optimizer states, and the process of updating every parameter demands substantial computational power, making it challenging to execute without specialized hardware [24, 25].

### 2.6.2 Partial Fine-tuning

To reduce the computational cost in full fine-tuning, partial fine-tuning strategies update only a subset of the model's parameters while keeping the rest frozen. This

approach is based on the observation that not all layers in a pre-trained model contribute equally to the performance of a specific task. In practice, this can be implemented with simple heuristics, such as unfreezing only the last few layers of the model, or through more sophisticated methods that analyze the model to identify the most influential layers for the given task. By freezing a significant portion of the network, partial fine-tuning significantly lowers the number of trainable parameters, leading to faster training times and reduced memory requirements. This makes it a practical alternative in resource-constrained environments, often achieving performance that is competitive with full fine-tuning [26].

### **2.6.3 Parameter-Efficient Fine-Tuning**

Parameter-Efficient Fine-Tuning (PEFT) methods further reduce the computational burden by freezing the entire pre-trained model and augmenting it with a small number of new, trainable parameters. One prominent PEFT technique is QLoRA (Quantized Low-Rank Adaptation). This approach involves two key innovations: model quantization and low-rank adapters. First, the weights of the frozen pre-trained model are quantized to a very low-precision format, such as 4-bit, which drastically reduces the memory needed to store the model. Second, small, trainable modules known as “adapters” are inserted into the model’s architecture. During the fine-tuning process, only the parameters of these adapters are updated, while the millions or billions of parameters in the base model remain unchanged. Gradients are backpropagated through the frozen, quantized model to optimize the adapter weights. This combination makes it possible to fine-tune extremely large models (e. g., over 60 billion parameters) on a single consumer GPU [24].

## 3 Context

### 3.1 Garden Linux

Garden Linux is an open-source operating system developed by SAP, designed with a strong focus on security hardening and customizability. It is a Debian-derived distribution that, by default, ships only essential functionality which can be extended as needed. This reduces the attack surface significantly. Overall, this makes the system an ideal operating system for the seamless execution of containers [27].

The system is designed to be highly secure. Therefore, it strictly implements security best practices, such as a reduced package set, hardened kernel configurations, and regular security updates [27].

To achieve security standards, such as FIPS 140-3 compliance, Garden Linux must ensure that all cryptographic functions used within its source code and open-source dependencies are analyzed and certified [4]. This requirement motivates the development of a tool that helps automate the identification of cryptographic implementations.

### 3.2 FIPS Relevance for Garden Linux

Garden Linux depends on many external open-source packages that might introduce cryptographic components that need to be considered for FIPS certification [27]. In open-source environments, the implementation of cryptographic functions may differ between packages instead of using a central cryptographic module. They can appear in libraries, system services, command line tools, and other dependencies that become part of the final operating system image. Garden Linux must be able to identify which components rely on unvalidated implementations [4].

This is especially important for self-implemented or copied cryptographic functions. If a dependency contains its own implementation of a cryptographic function instead of calling a standardized and reviewed cryptographic library, this code cannot automatically inherit the assurance provided by the validated library [4]. For Garden Linux, each of these implementations increases the effort required for analysis, documentation, and possible certification on the path to FIPS 140-3 compliance. Consequently, detecting

self-implemented cryptographic functions is directly relevant for reducing certification effort.

### 3.3 Related Work

The identification and analysis of cryptographic algorithms within software has been studied in recent years. These approaches cover simple pattern matching, machine learning techniques, usage of LLMs, and dynamic analysis of software during execution.

#### 3.3.1 CryptoGo

CryptoGo, a project developed by Li et al., focuses on detecting cryptographic API misuses, specifically within the Go ecosystem. The authors predefine function-specific rules that indicate cryptographic API misuses. These rules are solely based on the algorithm used and the arguments the API takes. Therefore, the authors define general rules like “Do not use insecure cryptographic algorithms” (like MD5) and specific rules like “Do not use the padding PKCS1-v1.5 for RSA”. In total, 12 rules are defined to track cryptographic API misuses in Go source code [28].

The authors use static taint analysis on Intermediate Representations to track data flow between sources (e.g., untrusted inputs or configurations), propagators (e.g., variables and function arguments), sanitizers (functions that make untrusted variables trustworthy), and sinks (cryptographic API calls) [28].

While very effective for its use case, it is strongly limited to identifying misuses of established APIs. Therefore, their work does not detect any custom implemented cryptographic logic. However, the manner in which rules are defined and used in static taint analysis can be an applicable use case for detecting self-implemented cryptographic functions in Garden Linux [28].

#### 3.3.2 CryptoGuard

Similar to CryptoGo, the CryptoGuard project introduces a tool that detects cryptographic API misuses in Java code, while focusing on reducing the amount of false positives. CryptoGuard defines a total of 16 vulnerabilities which can be detected in Java source code. It employs a static analysis approach that extracts only parts of the code which are relevant to identify cryptographic API misuses, such as exposed secrets and predictable pseudorandom number generators [29].

To prevent high false positive rates, the authors introduce refinement algorithms that identify and filter out security irrelevant elements such as constants with type definitions that are incompatible with the analysis context (e. g., booleans can't be used as keys). The authors claim to have reduced the number of false alerts by 80% for large-scale Apache projects and Android apps, but also note that they introduced false negatives caused by their refinements [29].

Like CryptoGo, CryptoGuard scans source code for misused cryptographic APIs, which is not the goal of this work. Still, their source code slicing techniques can also be used in the use case of detecting self-implemented cryptographic functions. A tool that detects such implementations to acquire FIPS certification should prefer the detection of false positives rather than false negatives, but the methods that are used to reduce false positives can still be applied to this work for rules where the risk of increasing false negatives is not affected. For example, after FIPS certification, a code pipeline could be introduced, which regularly scans every code change for possible self-implemented cryptographic functions. Reducing false-positives in such scenario can be useful to reduce developer workload.

### 3.3.3 Detection of cryptographic algorithms with grap

Léonard Benedettia et al. introduced “Detection of cryptographic algorithms with grap”, which explains a tool that detects specific code patterns in binary files. Grap uses a technique based on the analysis of the code's Control Flow Graph (CFG). Compiled binary files are disassembled into their respective CFG and analyzed for patterns of specific cryptographic functions. These patterns are represented by a flow of instructions, which must be constructed for each cryptographic function that should be made detectable [30].

The detection does not rely on constants, which helps in case these constants have been slightly modified or are not readable. In addition, the authors claim to achieve relatively fast detection, providing an example of processing the LibreSSL library, which took about 9 seconds to disassemble and perform the first pattern match [30].

However, their approach requires to create precise patterns of cryptographic algorithms, which essentially requires to understand and study the specific algorithm. They also mention that slight differences in implementation, compiler parameters and disassembler options can result in totally different CFGs. Therefore, designing a generic pattern that matches the same algorithm in different projects can be tricky [30].

The authors created signatures of the AES and ChaCha20 algorithms and claim to detect these algorithms in various compiled versions of LibreSSL. Every additional cryptographic function that Garden Linux aims to detect would need to be manually constructed. Still, this work creates a good baseline on how to convert binaries into CFGs and explains which features could be considered for detection of cryptographic functions [30].

### 3.3.4 Aligot

The Aligot project uses a dynamic analysis approach to identify cryptographic functions in binary files by observing their input-output relationship during execution. Because cryptographic functions should produce unique transformations of their inputs, this tool can identify primitives by matching these observed input-output behaviors against known cryptographic algorithms [31].

Their main advantage, compared to the other projects mentioned, is that Aligot bypasses structural obfuscations of algorithms. Additionally, there are only very few edge cases where false-positives can occur. If an output matches the result of an input, which is processed using a specific cryptographic function, it is safe to assume that this specific function is implemented. Aligot argues that the execution of most cryptographic functions contains loops. Hence, possible candidates for input and output variables are found by identifying loops within execution traces and capturing their data flow [31].

However, the dynamic analysis approach comes with two huge drawbacks. First, the analysis process requires the application to be executed while every execution step is traced. Executing the application is not always possible and it is difficult to test every feature of that application, especially in the case of libraries in Garden Linux. Second, after tracing every step from the application, numbers that could be the result of a cryptographic function must be entered into every cryptographic function that requires detection. While Aligot claims to process this faster compared to other dynamic approaches, it still does not scale well [31].

While not entirely suitable for the Garden Linux use case, their work covers the importance of loops in cryptographic functions and how they can be handled for detecting such methods.

### 3.3.5 Identification of cryptographic algorithms in binary programs

This thesis by Pierre Lestringant covers the detection of cryptographic algorithms in binary applications. Their work is motivated by concerns about developers implementing insecure cryptography into software, either by using insecure algorithms, using problematic input parameters or implementing these algorithms such that they can leak sensitive information [32].

Prior to analysis, the binary application is executed while its assembly code is traced. The code is then constructed into a normalized Data Flow Graph (DFG) using techniques like constant folding and memory access simplification. To analyze the DFG for cryptographic implementations, they differentiate between cryptographic primitives and “Mode of Operation”. Cryptographic primitives are standalone algorithms that represent the foundation of cryptography while “Mode of Operation” algorithms depend on said primitives. For example, the SHA256 algorithm is considered a primitive while HMAC-SHA256 can be considered a “Mode of Operation” algorithm. Primitives are detected using a signature based approach while “Mode of Operation” algorithms need manual interpretation based using the context around the primitive being used [32].

Additionally, their work includes valuable information about cryptographic algorithms that can be used for analysis. For example, the author introduces the “Straight-Line Code Hypothesis” for cryptographic functions, stating that such algorithms barely implement any conditional branches. Furthermore, they claim that Single Instruction Multiple Data (SIMD) instructions occur relatively often in implementations of cryptographic primitives and that “Mode of Operation” implementations often use the `memcpy` function in C. A SIMD instruction can perform an operation such as the `XOR` of two vectors (e.g., `[a0, a1] XOR [b0, b1]`) in a single instruction. Also, their work teaches how to construct DFGs from code and how to normalize them for better comparison of different implementations of the same algorithm [32].

### 3.3.6 CryptoLLM

The CryptoLLM project represents the shift toward using LLMs to identify complex cryptographic API misuses. The authors of the project trained their detection model based on  $\approx 40,000$  Java code snippets containing secure cryptographic API uses and  $\approx 55,000$  Java code snippets containing cryptographic API misuses, which are categorized in 5 categories. These code snippets have been collected from  $\approx 80,000$  Android projects using three steps [33].

First, crypto lines are extracted based on 5 cryptographic APIs that could cause vulnerabilities. Second, the context around these API calls is captured using a tool called **COMEX**, which generates the CFG and DFG corresponding to the Java file. The relevant code segments are identified by exploring the graphs for parameters that affect the API call and may cause vulnerabilities, including if-statements with a depth of 1 if necessary. In the last step, unnecessary features and code are removed from the code segment to achieve generalization. This includes comments and user defined function or variable names [33].

After collecting and manually labeling the data, the authors fine-tuned a LLM based on the dataset. For fine-tuning, they make use of a “binary cross-entropy loss function” which enables binary classification into benign API usage and API misuse. While not explicitly mentioned in their work, using this classification method typically modifies the weights in the model and introduces a classification layer, such that the model yields a classification estimation. The trained model can then be used to automatically classify other code snippets from yet unreviewed projects [33].

This project introduces a promising approach ( $> 94\%$  accuracy and precision) to detect misuses of cryptographic APIs using a non algorithmic way, which enables the classification of syntactically modified code snippets. While Garden Linux does not aim to identify misuses in cryptographic API calls within their libraries used, the classification using LLMs may also be used to identify self-implementations of cryptographic functions. However, the challenge of capturing the necessary context persists for Garden Linux. While the CryptoLLM project identifies possible candidates using specific Java API calls, such definite code identifiers do not exist for self-implemented cryptographic methods [33].

### 3.3.7 CryptoScope

Li et al. introduce CryptoScope, a tool that aims to fill the gap of missing detection of flaws in cryptographic algorithm implementations. They use a large dataset of over 12,000 cryptographic related texts, crawled from various sources like online forums and research papers. This data represents a knowledge base of cryptography in general which the detection model later uses to identify and reason flaws in implementations [34].

Implementations of cryptographic functions are fed into an advanced LLM (e. g., OpenAIs GPT-4). In case the algorithm is already standardized, the LLM checks conformity using the information provided by the FIPS standard. For not standardized algorithms,



CryptoScope employs Chain-of-Thought (CoT) reasoning to analyze the underlying cryptographic intent of the algorithm. To reduce false positives, false negatives and insufficient analysis, the authors implement a Retrieval-Augmented Generation (RAG) system which provides the LLM access to the previously mentioned 12,000 cryptographic texts [34].

In the context of Garden Linux, this tool can be a great support for code auditing tasks, as it allows to explain implementations of cryptographic functions and indicate whether they require modification. Nonetheless, the goal of this thesis is to implement a detection model of self implemented cryptographic functions, deciding whether these are insecure is out of scope for this work.

### 3.3.8 FoC

FoC (Figure Out the Cryptographic Functions) is a framework designed to identify and analyze cryptographic functions in stripped binary files. The project uses compiled versions of established libraries like OpenSSL and Libgcrypt as a baseline to build their detection model. Using decompilers, these binaries are reverse engineered into pseudo C code. To increase the data volume, the authors use techniques like compiling with different compilers and parameters, leading to more than 1.8 million unique crypto-related functions. For each function, a LLM creates a short summary as a label that represents the function's semantics [35].

The framework consists of two main components [35]:

- FoC-BinLLM, a LLM model fine-tuned on source code, pseudo code and their labels. This allows the model to formulate function semantics in natural language
- FoC-Sim, a similarity model that utilizes code embeddings, graph structure information via Graph Convolutional Networks, and handcrafted cryptographic features. This model uses the semantics encoded by FoC-BinLLM and a Graph Convolutional Network (GCN) to embed functions into a vector that can be compared with their large database.

Similar to other LLM-based approaches, FoC demonstrates high performance in semantic summarization and similarity detection. The primary goal of FoC is algorithm identification which aligns with the goal Garden Linux is aiming at. While Garden Linux works with open source libraries, where the source code can directly be used for analysis, the framework's methodology provides a strong architectural reference for this thesis [35].

### 3.3.9 Automated Recognition of Cryptographic Algorithms via Machine Learning

This work proposes an automated recognition system that employs machine learning techniques to identify cryptographic algorithms based on their outputs. Their work uses a dataset of encrypted texts from popular encryption algorithms (e. g., AES and RSA) used to train a machine learning model to classify ciphers [36].

The authors use 6 different machine learning classifiers which are trained to distinguish between various encryption algorithms using only the encrypted text. They tested a Support Vector Machine, Decision Trees, Random Forests, K-Nearest Neighbors, a Convolutional Neural Network (CNN) and a Multilayer Perceptron. Their experiments show, that CNNs are best at classifying ciphers with an accuracy of 97% [36].

While their work does not cover the detection of cryptographic functions using features in code, it is included in this chapter as it can be used as a legitimate way of detecting which cryptographic function has been used inside a software.

### 3.3.10 Pattern Recognition using LLMs

Dozono et al. present an empirical study on the applicability of LLMs for secure code assessment across five programming languages. Their work covers the two tasks of binary vulnerability detection and CWE classification. For this, they compiled a manually validated dataset of 378 code snippets that is balanced between vulnerable and non-vulnerable examples and covers the top 25 CWE categories. [37]

The authors evaluate six LLMs (GPT-3.5-Turbo, GPT-4 Turbo, GPT-4o, CodeLlama-7B, CodeLlama-13B, Gemini 1.5 Pro) under various prompt strategies. Their experiments show that larger and more capable models, like GPT-4 Turbo and GPT-4o, generally achieve the best detection and classification scores. However, they do not report monetary pricing for model usage [37].

For Garden Linux, the study highlights a relevant observation. While the study does not directly detect cryptographic implementations, it proves that LLMs have strong pattern matching capabilities that may also be applicable to the detection of self-implemented cryptography in source code.

### 3.3.11 Comparison of Related Work

The features of all tools explored in the earlier sections can be compared in a table.

Figure 4: Tabular comparison of related work

| Source             | Detects Self-Implementations | Dataset                                                                                             |
|--------------------|------------------------------|-----------------------------------------------------------------------------------------------------|
| CryptoGo [28]      | No                           | 120 open-source Go projects, focusing on API misuse.                                                |
| CryptoGuard [29]   | No                           | 46 large-scale Apache Java projects and 6,181 Android apps, focusing on API misuse.                 |
| grap [30]          | Yes                          | Compiled libraries (LibreSSL, libsodium) with different compilers and optimizations.                |
| Aligot [31]        | Yes                          | Obfuscated synthetic programs and real-world malware.                                               |
| Lestringant [32]   | Yes                          | Implementations from OpenSSL, LibTomCrypt, etc., compiled with GCC/Clang and various optimizations. |
| CryptoLLM [33]     | No                           | 95,000 Java snippets from 80,000 Android projects, focusing on API misuse.                          |
| CryptoScope [34]   | No                           | 92 samples from CVEs, CTF challenges, synthetic examples.                                           |
| FoC [35]           | Yes                          | Custom dataset from popular C-based crypto libraries.                                               |
| Shree et. al. [36] | No                           | Dataset of encrypted texts from various algorithms (e. g., AES).                                    |
| Dozono et al. [37] | No                           | Multi-language dataset (378 snippets) across Python, C, C++, Java, JavaScript                       |

The table above compares the related work. It evaluates each approach against its capability to detect self-implemented cryptographic functions and the dataset it was evaluated on. This comparison shows a clear distinction between two main categories.

The first category, which includes CryptoGo, CryptoGuard, and CryptoLLM, focuses on identifying the misuse of well-known cryptographic APIs. These tools demonstrate high accuracy and precision on large-scale codebases, but they are not designed to detect self-implemented or custom cryptographic logic. Additionally, the CryptoScope tool enhances LLMs to detect vulnerabilities and logic flaws in implementations of cryptographic functions. Therefore, these tools are not directly applicable to the primary goal of this thesis.

The second category is more aligned with the objectives for Garden Linux. Tools like *grap*, *Aligot*, Lestringant's thesis and *FoC* are designed to identify cryptographic algorithms based on their implementation details, not only in source code, but also in binary. These approaches vary a lot. *Grap* and Lestringant's work rely on manually created signatures of code or data flow graphs, which can be very precise but require significant expert effort for each new algorithm. *Aligot* uses a dynamic input-output analysis, making it more robust to code obfuscation but difficult to apply in a static context and is potentially slow. More recent approaches like *CryptoScope* and *FoC* apply LLMs to understand and classify the semantics of the code itself.

The work by Shree et al. stands apart, as it classifies algorithms based only using their ciphertext output, without analyzing the code that produced it. While achieving high accuracy, this method does not help in locating the cryptographic implementation within the Garden Linux codebase.

Finally, the work by Dozono et al. is unrelated to detecting cryptography in its core form, but it suggests that chat LLMs like ChatGPT can be leveraged to classify source code as containing crypto or not.

## 4 Methodology

Based on the related work presented in Section 3.3, this thesis presents a model based on vector embeddings to detect implementations of cryptographic functions in software. Code snippets are embedded into high-dimensional vectors and then classified using binary classifiers. To train, test and evaluate these models, a large dataset of self-implemented cryptographic functions and negative examples is needed.

### 4.1 Dataset Construction

To train and evaluate the detection model for identifying custom cryptographic functions, a diverse dataset of source code files is constructed.

#### 4.1.1 Dataset Requirements

The dataset constructed for this thesis intentionally differs from datasets used in related work (see Section 3.3). These studies target different goals than this thesis, primarily the detection of vulnerabilities in cryptography used in software. Therefore, their datasets are not directly applicable to this thesis and the dataset used here is constructed with the following requirements:

- **Self-Implementations:** Files are intentionally not chosen from well-known cryptographic libraries. This differs from the way many studies in Section 3.3 construct their datasets. Those studies often rely on copies of well-known cryptographic libraries or synthetic function snippets. While such sources are useful for detecting any use of cryptography in general, they do not represent the variety of self-implemented cryptographic code as relevant for Garden Linux. Therefore, another approach is chosen where files are collected from a wide range of public repositories to capture different coding styles.
- **C and C++ source code:** While many projects mentioned in Section 3.3 aim to analyze compiled binary files, Garden Linux always has access to the source code of used dependencies. Thus, the dataset is constructed with source code only. The programming languages C and C++ dominate Garden Linux dependencies and are therefore the most important files for the target use case.
- **Representative Negative examples:** Include files that use cryptographic APIs (like OpenSSL, libsodium, etc.) as explicit True Negative examples. This prevents the

model from learning that the mere presence of crypto keywords implies a self-implementation.

Following these requirements helps constructing a dataset well suited for detecting self-implemented cryptography in source code. The requirements address potential biases towards source code that make use of cryptographic APIs and source code that implement modified versions of known cryptographic algorithms.

#### 4.1.2 Data Gathering and Filtering

A Python script performs a systematic crawl of publicly available repositories on GitHub, targeting C and C++ files that contain common cryptographic identifiers. Each query utilizes one of the following keywords associated with standard cryptographic algorithms and primitives:

```
1 AES-GCM, DES, 3DES, Blowfish, Twofish, Camellia, RC4, ChaCha20-Poly1305,
  ECDSA, Ed25519, RSA, ECDH, MD5, SHA-1, Poly1305, SHA-256, SHA-512, HMAC-
  SHA256, BLAKE2s, x25519
```

Using these keywords, a total of 988 source code files is collected. Note that this database is only constructed with files mentioning known cryptographic algorithms. When training a detection model based on this data, cryptographic functions not included in the labeled dataset and self-invented cryptographic functions may stay undetected. Also, it is important to note that not every file that contains any of these keywords is actually using the respective cryptographic function. For example, a lot of files mention these keywords only in code comments (e.g. “This function compares two SHA-256 hashed values”).

#### 4.1.3 Labeling

The raw dataset is refined through a two-step filtering process:

- 1. Manual Sanitization: Files clearly utilizing standardized libraries (like OpenSSL) are manually identified and labeled as such.
- 2. LLM-Assisted Annotation: As the author’s knowledge of the cryptographic algorithms mentioned above is not sufficient for manual classification, the “Claude Sonnet 4.6” language model is assisting the process. The model is prompted to justify whether a code file uses “custom cryptographic implementations,” “code using cryptographic APIs,” or “files completely unrelated to cryptography.” Using

the model’s description and the source code, the author decides to either accept or decline the model’s suggestion.

This process results in:

- **285** files containing custom cryptographic implementations.
- **480** files identified as cryptographic implementations relying on standard API usage.
- **223** files which are entirely unrelated to cryptographic functions.

These 988 files are further separated into training, validation and test datasets as suggested for the Three-Way Holdout Method mentioned by Sebastian Raschka [13]. Both, validation and test sets, should contain 15% of the whole dataset each. To ensure Stratification, the validation set is therefore built using 43 self-implemented cryptographic files, 72 files that rely on standard API usage and 33 unrelated code files. Then, multiple validation and training datasets are created from the remaining files. While the validation datasets consist of the same size and distribution like the test set, the training dataset holds the remaining 199, 336 and 157 files from these categories, respectively.

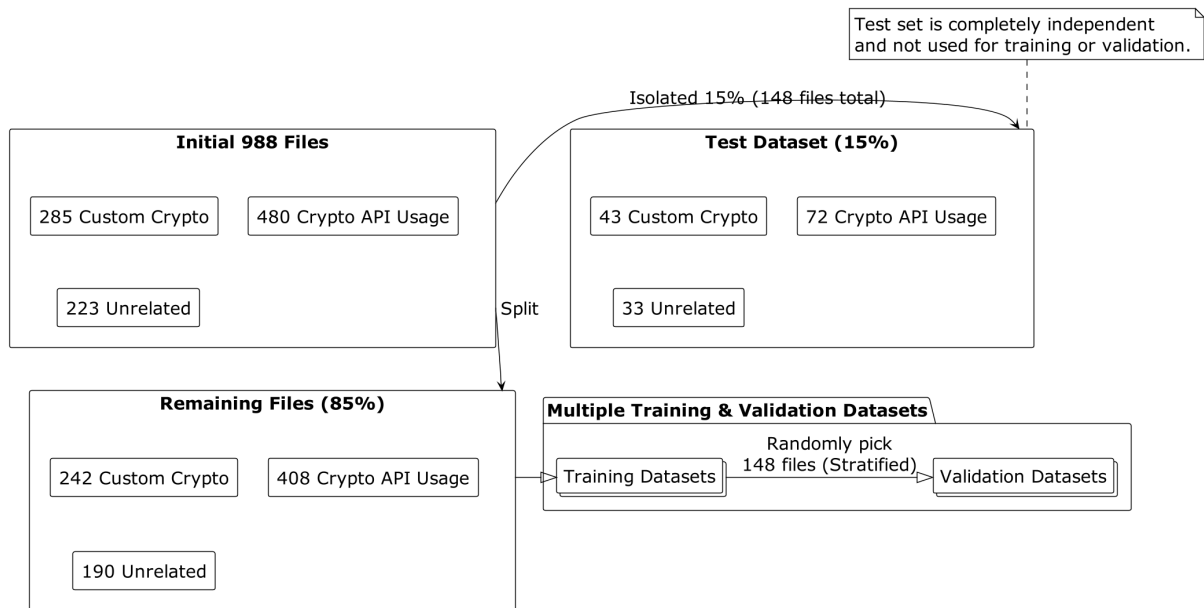


Figure 5: Visualization of the dataset splitting process. The test dataset is completely separate, and validation datasets are randomly sampled from the training data while maintaining stratification.

The datasets are used to train a model with the task of classifying files into “contains self-implemented crypto” or “does not contain self-implemented crypto”. Hence, one might wonder why the datasets show a distinction in 3 subclasses instead of two.

Training sets of models should always be representative subsets of the actual real world distribution [13]. Training a model only on files unrelated to crypto does not guarantee that the model performs well when confronted with files that contain cryptographic API usages. Similarly, training the model primarily on files with cryptographic API usages does not guarantee, that the model can reliably distinguish code unrelated to cryptography and self-implementations of cryptographic functions. In theory, this could be extended further by considering other classes like, for example, code that solves mathematical equations.

The distinction allows to further evaluate the model and make assumptions of how good the model performs regarding a specific category. For example if the model would yield an accuracy of 0.78 ( $\approx (\frac{43+72}{43+72+33})$ ), one might think that the model performs relatively well. However, when viewing this metric while ignoring the counterpart subclass, values like  $accuracy_{\text{without\_api\_crypto}} = \frac{43+72}{43+72} = 1$  and  $accuracy_{\text{without\_non\_crypto}} = \frac{43}{43+33} \approx 0.57$  could emerge, indicating that the model successfully learned to distinguish crypto code from code that uses crypto APIs while it completely fails to distinguish crypto unrelated code and self-implemented cryptographic functions. In this scenario, the model only learned to distinguish code that uses crypto APIs. Any other code would be wrongly classified as “self-implemented crypto”. Hence, this subclass separation can also be used to identify flaws in the model.

#### 4.1.4 Dataset Limitations

The dataset constructed in this work is not representative for general open source dependencies. It is constructed exclusively from files that somehow mention a cryptographic algorithm in code, comments or file name. This applies even for files that are labeled “unrelated to crypto”. In addition, the negative group consists of  $\approx 70\%$  files that use crypto APIs. This distribution differs significantly from realistic open source projects. It is hard to tell the exact realistic distribution, but it is very likely, that a representative dataset would consist of a large amount of files that are completely unrelated to cryptography, such that cryptographic algorithms are not even mentioned in code or comments.

In this work, the dataset is constructed with unrealistic distribution because labeling a very large amount of negative cases would require too many resources (e. g., time). The consequences are manageable:

- 1: Test results on the dataset are not directly applicable to real-world scenarios.



- 2: The resulting detection tool should also be tested on a real-world scenario.
- 3: Comparisons with other projects should be conducted on their datasets to ensure fairness.

There is another important constraint that must be noted. As mentioned earlier, file labeling is assisted by the “Claude Sonnet 4.6” language model by providing summaries and recommendations as the author’s expertise in this matter is limited. However, there is no guarantee that such language models provide correct results. In fact, studies show that LLMs may fall for traps that human non-experts would fall for too [38]. Thus, this labeling method is prone to mislabeling of files that are difficult to sort into a group.

For this reason, a manual extensive analysis of 35 files, independent of LLM suggestions, is conducted. Details about this experiment can be found in Appendix 2. All 35 randomly chosen files seem to be correctly classified in the dataset. While this does not guarantee that all files in the dataset are labeled correctly, the absence of mislabeled files in the randomly chosen files provides evidence that the mislabeling rate is relatively low. The “Rule of Three” [39] gives a 95% confidence that the upper mislabeling rate is at  $\approx 8.6\%$ .

These findings suggest that the sole use of strong chat models like “Claude Sonnet 4.6” are sufficient for the goal that Garden Linux aims for. However, this comes with economic and environmental constraints that are further discussed in Section 5.5 and Section 5.6.

## 4.2 Evaluation Metrics

In Section 2.3, 3 metrics were introduced which can be used to evaluate the performance of a classification model. The Precision metric indicates how good the model can right-fully classify code as crypto ( $\text{True Positive} / (\text{False Positive} + \text{True Positive})$ ). The Recall metric is used to identify how strong a model performs at classifying crypto code snippets as such ( $\text{True Positives} / (\text{True Positives} + \text{False Negatives})$ ). Lastly, the F1-Score is a Metric used to indicate how well the model performs in general, by considering both other metrics in its calculation ( $2 * \text{Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$ ).

In other words, Precision should be maximized if the model’s main goal were to prevent non-crypto functions from being flagged as crypto, which would introduce redundant workload. On the other hand, maximizing Recall prevents crypto functions

in the codebase from staying undetected. Both scenarios have valid use cases in the context of detecting self-implementations of cryptographic functions in Garden Linux dependencies.

A model with high Precision could be integrated in a pipeline, where it is executed after any code change in the Garden Linux environment and scans for potential introduced self-implemented crypto functions, while accepting the risk for more false negatives. Implementing a model with high Recall can be useful for tasks where an entire codebase has to be audited. The model yields much fewer False Negatives than a model optimized for high Precision, but it generally also introduces much more False Positives. Still, redundantly auditing some False Positives can be much faster than having to audit every single code file. For this to work, the model needs to confidently detect every actual self-implemented cryptographic function in the codebase. However, even if the detection model does detect every single cryptographic function in the testing dataset, there is no way to guarantee that this also applies for yet unseen data.

As both metrics, Precision and Recall, tend to be important for evaluating model performance, the F1-Score can be used as an “overall performance” metric, which is especially useful for model comparison. In this work, several different classification models are trained. In cases where these models are compared, the F1-Score will be used. Later, when validating the final model and when it is compared to other models, other metrics may be used that fit the respective context.

### 4.3 Model Pipeline

The detection model structure can be understood as a pipeline consisting of preprocessing, embedding, and classification.



Figure 6: Pipeline of the detection model’s training process.

In the first step, the source code files need to be sliced into small chunks. The actual size of these chunks depends on the embedding model used. For example, the CodeBERT LLM is limited to an input size of 512 tokens [22], which means that the source code files must be sliced, such that every chunk can be fed into the embedding model. Files are first translated into tokens by the model’s tokenizer and then chunked with a predefined overlap to the next chunk. This overlap ensures that context (e. g., functions)

is preserved in a single chunk and not split among multiple pieces. A 512 token chunk usually includes around 20 to 50 lines of code, depending on the amount of words and symbols per line.

Next, the chunks are fed into an embedding model which results in high-dimensional vectors for each chunk. For example, the CodeBERT model yields embedding vectors with 768 dimensions [22]. This dimension size is later relevant for the classification models.

This is followed by the dataset preparation which adapts the concepts introduced in Section 4.1.3. Hence, data is split in individual training and validation datasets while making sure every subclass (crypto, non-crypto, crypto-api usage) evenly represents the original dataset. Based on this, every embedded chunk is labeled accordingly as originating from the training, validation or test dataset and as being from a crypto, non-crypto or crypto-api file.

Using the labeled data, a binary classifier model can be trained to differentiate embeddings of cryptographic functions from others. Here, two different classification models are tested based on the prior research. The CryptoLLM project mentioned in Section 3.3.6 suggests using a classification method based on a fine-tuned LLM which has been enhanced with a “binary cross-entropy loss function”. This is basically a neural network trained to differentiate between two classes. Similarly, OpenAI suggests training a random forest classifier (as introduced in Section 2) to classify vector embeddings [40]. In the last step, the performance of the model is evaluated.

## 4.4 Hyperparameter Tuning

The detection model as introduced above comes with a lot of tunable parameters which can be addressed to influence the model’s performance. There are 9 parameters that will be tuned in this work.

Embedding related parameters:

- 1. The embedding model used
- 2. The chunk overlap size

Neural Network classifier parameters:

- 3. Amount of epochs used for training
- 4. Size of the batches used in training
- 5. Learning rate applied in training

- 6. Architecture of the Neural Network

Random Forest classifier parameters:

- 7. Amount of trees in the forest
- 8. Maximum depth of trees
- 9. Minimum amount of samples needed to split a leaf

Theoretically speaking, there are more parameters that can be tuned to increase the model performance. However, any additional parameter introduces exponentially increasing computing time. In this scenario, time is a very limiting factor when fine-tuning parameters, especially if the chunks have to be embedded multiple times with different embedding models and chunk overlap sizes.

Note that the most impactful hyperparameter, the decision threshold, is not tuned manually. Classifiers always yield a prediction score between 0 and 1. Whether a file is classified as containing self-implemented crypto or not is decided upon the decision threshold. It is easy to mathematically compute the best threshold for each classifier after all predictions are obtained. Therefore, during training, always the best threshold of a classifier is used to calculate its evaluation metric. For the final model, a fixed threshold must be applied.

#### **4.4.1 Embedding and Classifier parameters**

Three diverse embedding models are tested in this work:

| Model Name                               | Description                                                                                                                                 | Token Limit                                  | Size on Disk |
|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|--------------|
| microsoft/codebert-base [22, 41]         | BERT-based encoder pre-trained on natural language and source code for code search, documentation generation, and code understanding tasks. | 512                                          | 477 MB       |
| Alibaba-NLP/gte-Qwen2-1.5B-instruct [42] | Instruction-tuned text embedding model based on the Qwen2.5 1.5B architecture. Produces dense embeddings rather than generative text.       | 32,768 (Realistic: 4096 to fit on 12GB VRAM) | 6.7 GB       |
| Qwen/Qwen2.5-7B-Instruct [43]            | Instruction-tuned large language model optimized for conversational reasoning, code generation, and general-purpose text generation.        | 32,768 (Realistic: 2048 to fit on 12GB VRAM) | 15 GB        |

Table 3: Comparison of evaluated language models

This diverse selection of embedding models ensures that a good balance between computation time and model performance can be determined. Both models, the Qwen 2.5 7B and the Qwen 2.5 1.5B model, support a token limit of 32,768. However, the attention mechanism consumes a lot of memory as the sequence gets longer. In practice, this means, when setting the chunk size to 32,768 tokens, the embedding process runs out of memory on an NVIDIA RTX 3060 with 12GB VRAM. Thus, chunk sizes are set to 4096 and 2048 tokens as they have been identified as ideal token limits for this specific hardware. A token limit of 4096 leads to an average of 3 chunks per file, which is probably enough to cover any important cryptographic context.

After evaluation of the three embedding models is done, the best chunk-overlap is determined. For this, two different overlap configurations are considered:

| Proportion | CodeBERT   | Alibaba     | Qwen        |
|------------|------------|-------------|-------------|
| 1/8        | 64 Tokens  | 512 Tokens  | 256 Tokens  |
| 1/2        | 256 Tokens | 2048 Tokens | 1024 Tokens |

Table 4: List of Chunk Overlap Configurations tested.

These overlap sizes cover a good range of possible values. As mentioned before, experiments with the tokenizers of the CodeBERT embedding model show, that 512 Tokens usually cover 20 to 50 lines of code. Studies have shown, that an average function in C++ has a size of 19 lines, with a standard deviation of  $\sigma \approx 6$  [44]. Assuming similar function lengths apply to C code, a chunk size of at least 512 Tokens might be needed to reliably include every code function into a chunk as a whole. Alibaba’s Qwen model achieves this effortlessly and may even include context that is largely separated from functions, like constants that are declared way above the function that uses them. But while a higher overlap increases the chances of representing a whole function in a single chunk, the more chunks are generated in general. The more chunks that need to be analyzed, the higher the possibility for false positives. To observe this effect, both configurations are tested.

Lastly, the classifier parameters are fine-tuned.

|                                     |                                                |
|-------------------------------------|------------------------------------------------|
| <b>Amount of Trees (estimators)</b> | 50 - 250 (increment 50)                        |
| <b>Maximum Tree Depth</b>           | 3 - 15 (increment 3) and None                  |
| <b>Minimum Samples for Split</b>    | 2 - 7 (increment 1)                            |
| <b>Use Bootstrap</b>                | Yes (Allow OOBs) / No (Train on whole dataset) |

Table 5: Tested hyperparameters for the Random Forest classifier.

The hyperparameters for the random forest classifier are visualized in Table 5. The random forest will be trained on 50 to 250 amount of trees. While it is proven that larger forests tend to deliver better performance, as mentioned in Section 2.2.1, the improvement can be insignificant if too many trees are trained. If the evaluation yields that more trees only increase computing time without performing better, the smaller tree size can be chosen to save computation time.

Furthermore, the maximum tree depth parameter controls how deep a tree is allowed to grow. Smaller trees may be faster in computation and better in generalization, while longer trees are heavier to compute and risk to overfit. However, overfitting trees are often not causing the random forest to perform worse due to the large amount of trees and variance [6]. Thus, tree depths from 3 upward are tested to find a good balance for this parameter.

Another hyperparameter that can affect model performance is the minimum number of samples needed to decide whether an internal node is split or kept as a leaf. Similar

to a maximum tree depth, increasing the value of minimum samples needed for split can prevent overfitting of trees by preventing the tree from creating splits that are supported by only a few training samples [7]. This can be explained with the following example. If two samples from different classes follow the same path through the tree and are the only samples remaining in a node, if the node is split based on a feature that perfectly separates these two samples, each resulting leaf will contain only a single training example. Later in inference, if a new, unseen sample falls into one of those leaves, the model will confidently assign a class probability of 0 or 1, even though this decision is based on only one training sample. If the last split is not performed, both samples remain in the same leaf, and the predicted class probabilities reflect the class distribution in that leaf (e. g., 0.5).

The last hyperparameter considered in the fine-tuning process of random forest is whether the Bootstrap mechanism should be used to draw samples for each tree. As explained in Section 2.2.1 the bootstrap method draws a subset of samples with replacement from the whole dataset for each tree, which can leave OOBs that are not considered at all in the training of the random forest. Normally, this has the advantage that trees are constructed without correlation and, additionally, the OOBs samples can be used to evaluate the model. However, the OOBs are not guaranteed to evenly represent each classification, which violates the concept of “Stratification” introduced in Section 2.4.1 and hinders statistical comparison. Thus, the independent evaluation set as described in Section 4.3 is used for evaluation of the random forest. To prevent the random forest from learning on only a subset of the whole dataset, training will also be tested with bootstrap disabled.

|                                    |                                       |
|------------------------------------|---------------------------------------|
| <b>Number of Epochs</b>            | 50 - 250 (increment 50)               |
| <b>Batch Size</b>                  | 4, 8, 16, 32, 64                      |
| <b>Learning Rate</b>               | 0.00005 - 0.00045 (increment 0.00005) |
| <b>Neural Network Architecture</b> | 1 layer (Perceptron) / 3 layers       |

Table 6: Tested hyperparameters for the neural network classifier.

For the neural network classifiers, the hyperparameters tested are visualized in Table 6. The number of epochs define how many repeating training steps are run in the training process. While the model needs a general minimum of training epochs to understand the

underlying semantics needed for classification, too many epochs can cause the model to overfit [11, p. 109f].

The batch size determines how many training samples are processed before the loss is computed and the model parameters are updated [9]. A batch size of 1 causes the model to update its weights after every individual training sample. While this provides frequent updates, the resulting gradient estimates can be noisy, which can make training less stable and make it more difficult for the model to learn the underlying patterns that distinguish the classes [11, p. 276].

At each step in the training, the optimizer adapts the weights of the model using the predefined learning rate [10]. This hyperparameter can have a strong impact on model performance, which is why so many different learning rates are tested here. A higher learning rate can speed up the learning process as the weights adapt faster. However, higher learning rates can also prevent the model from reaching the optimum [11, p. 291].

Another parameter that affects model performance lies within the neural network architecture. Complex tasks require complex architectures with deeper layers [11, p. 242f]. However, it is unclear how complex this specific detection task actually is. Therefore, two different architectures are tested. One is a simple linear layer (Perceptron) with a dimension equal to the embedding vector dimension of the respective embedding model used. A linear layered neural network is visualized earlier in Figure 2.

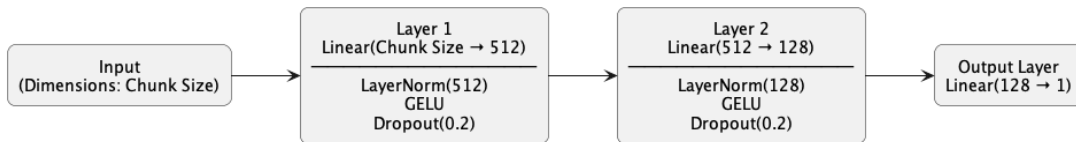


Figure 7: Architecture of the 3 Layer Neural Network Classifier

The second architecture is built on 3 layers and depicted in Figure 7. It is based on three features provided by the PyTorch library:

- Layer Normalization is a technique that normalizes the values produced by a layer so that they have a consistent scale. The key idea is that the neurons within a layer can produce outputs with very different scales during training. This can make optimization more difficult. Layer Normalization addresses this by computing the mean and variance of all the neurons in a layer for a single input sample. Each activation is then normalized by subtracting the mean and dividing by the standard deviation [45, 46].



- Gaussian Error Linear Unit (GELU) is an activation function that helps the neural network learn non-linear relationships in the data [47].
- Dropout: Dropout is a regularization technique used to reduce overfitting. During training, it randomly deactivates a fraction of the neurons. This prevents the network from relying too heavily on specific neurons. During testing or inference, all neurons are active [48, 49].

As described in Figure 7, the first layer takes the input features and transforms them into 512 neurons. After this, Layer Normalization stabilizes the values, the GELU activation function introduces non-linearity such that the model can learn complex relationships. The dropout randomly disables 20% of the neurons during training to reduce overfitting. The second layer reduces the representation from 512 neurons to 128 neurons, helping the model focus on the most important learned features. Again, Layer Normalization, GELU, and Dropout are applied. The final layer is an output layer that maps the 128 features to the classifier’s prediction.

#### 4.4.2 Running the Tests

The best parameters are determined by using every configuration mentioned in the section above and train a specific classifier model using a randomized training set. This yields a total of 4860 classifiers, trained on different embedding configurations and classifier hyperparameters.

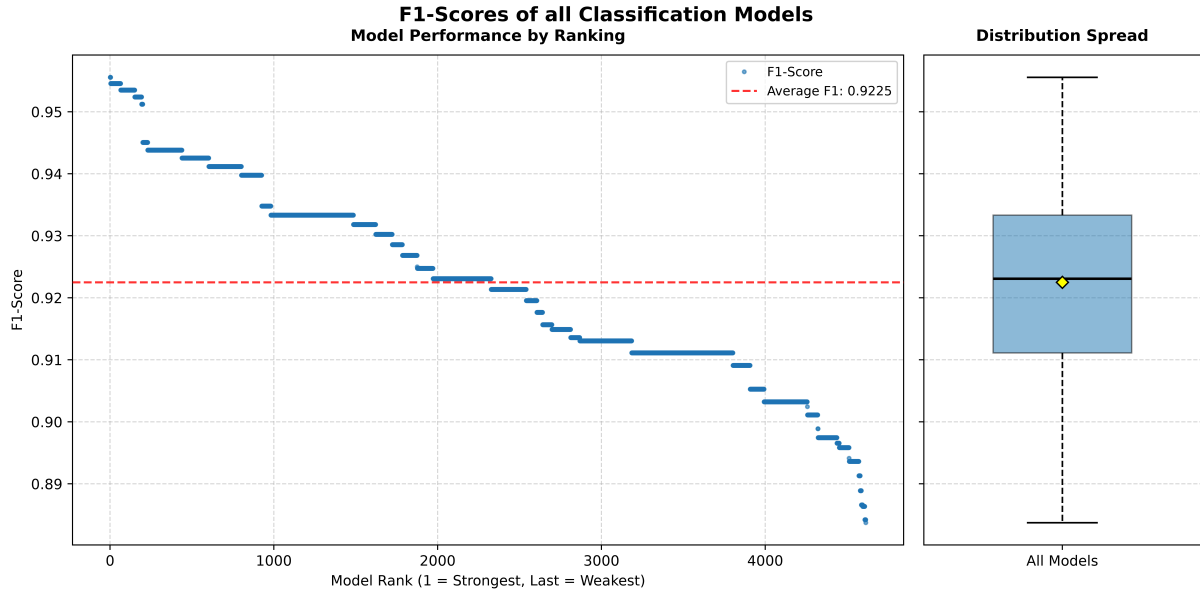


Figure 8: F1-Score distribution across 4860 classifier configurations. Each classifier's decision threshold is optimized to maximize its F1-Score, and all classifiers are consistently trained and evaluated on the same datasets.

The graph in Figure 8 shows all F1-Scores for all 4860 classifiers, each trained with a different configuration but on the same training and validation datasets. When optimizing the classifier's decision thresholds to maximize their F1-Scores, the F1-Scores range from 0.88 up to 0.96. While a score of 0.96 would indicate near perfect performance, without repeated testing, a single F1-score of 0.96 may not be representative of the model's true performance. For this reason, it can be better to directly compare the hyperparameter values instead of all specific configurations.

Comparing values of a hyperparameter can be achieved using the McNemar test as introduced in Section 2.4.3. The McNemar test is designed to compare the performance of two models on the same set of observations. Because of that, it is easily applicable to hyperparameters like the chunk overlap, as only two different values are tested. In order to compare models, the McNemar test considers all false positives and false negatives yielded by each model. Using the example of the chunk overlap hyperparameter, all 4860 classifiers can be distributed in 2430 classifiers that use a chunk overlap of  $\frac{1}{8}$  and 2430 classifiers that use a chunk overlap of  $\frac{1}{2}$ .

To determine whether the two chunk overlap values result in significantly different performance, all misclassifications yielded by configurations using an  $\frac{1}{8}$  chunk overlap are compared with all misclassifications of the  $\frac{1}{2}$  chunk overlap. McNemar performs a  $\chi^2$ -test on all misclassifications that both models don't agree on. The resulting  $p$ -

value represents the probability of both models being indistinguishable. In this work, a significance level of  $\alpha = 0.05$  is used. Consequently, a resulting  $p$ -value below 0.05 indicates that the null hypothesis of equal performance can be rejected, suggesting that the two hyperparameter values result in significantly different model performance. If two hyperparameter values yield statistically significantly different results, the hyperparameter value resulting in fewer errors is considered to perform better.

The same test can also be applied to all hyperparameters where multiple values are tested, like with the maximum tree depth in random forest classifiers. In order to get a ranking in this case, every value must be compared with all other values. A tabular visualization of the test results can be found in Appendix 3.

With more misclassifications and a resulting  $p$ -value of less than 0.01, the CodeBERT LLM proved performing worse than both other models, which is expected. Nonetheless, Qwen with its 7 Billion parameters does not seem to perform better than the modified Alibaba version with only 1.5 Billion parameters despite being larger. In fact, the comparison of both LLMs shows a  $p$ -value of  $\approx 0.04$ , while configurations using the Alibaba LLM yield less misclassifications. Additionally, the  $\frac{1}{8}$  chunk overlap performs better than the  $\frac{1}{2}$  chunk overlap.

Apart from that, it comes clear that the random forest classifier performs better in general than the neural network classifier does. This may be caused by the impurity of the data that used to train the classifiers. In the current workflow, the files are chunked, then every chunk is embedded and based on these embeddings, the classifiers are trained. For training, the embedded chunks are labeled as either crypto if originating from a file containing self-implemented crypto or non-crypto if otherwise. However, not all chunks that originate from a crypto file are guaranteed to contain crypto. Thus, it is possible that some chunks are labeled incorrectly while still being used for training the classifiers.

One possible explanation for random forests to perform better is that random forests could be more resistant than the neural network architecture when facing mislabeled chunks. While an incorrect chunk only affects some trees in a random forest that were trained on the wrong labeled data, an incorrect chunk affects a neural network's general performance as it adapts all its weights in a training step.

Other interesting insights are provided by the performance difference of the neural network classifier with 1 layer and 3 layers. With a  $p$ -value of 0.015, the 1 layered classifier proves being different from the 3 layered classifier while yielding higher error

rates. This indicates that the problem to be solved might be too difficult for a simple one-layered linear architecture.

Furthermore, the experiments show that, in random forests, maximum tree depths higher than 9 perform better. Also, applying bootstrapping when training the random forest seems to increase the classifier performance. The same applies to setting the amount of estimators to larger than 50. Any other hyperparameter tested does not show significant differences in their values.

Still, the question, which classifier configuration actually performs best, remains. While some values of one parameter seem to perform better than others, the strongest model is not necessarily defined by the combination of the best values of all hyperparameters. Therefore, additional steps are done to find the best performing configuration among all. Using the bootstrap method to compare different models with each other introduced in Section 2.4.1, the strongest configuration is determined. The bootstrap size is set to 692 (the size of a training dataset) and each classifier is trained 40 times on different bootstrap samples. To reduce runtime and prevent unnecessary tests, only the strongest 50 configurations from Figure 8 are used for comparison. Among the strongest configurations, only random forest classifiers appear. Thus, no neural network classifiers are tested from this point on. Additionally, all classifiers are either built on embeddings provided by Qwen or Alibaba-NLP. Therefore, Microsoft CodeBERT is not tested anymore.

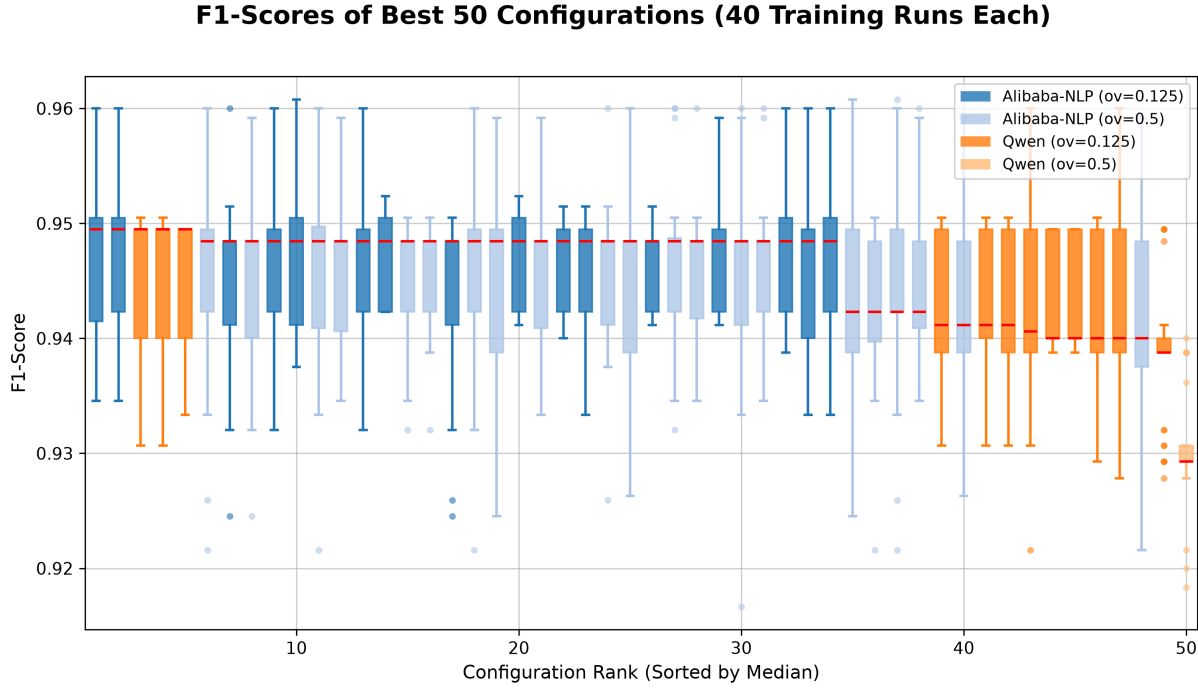


Figure 9: Boxplots of F1-Scores across 50 classifier configurations. All classifiers are trained and evaluated on 40 different training and evaluation datasets.

The data yielded by these 2000 evaluations shows little variance. All medians are situated between 0.92 and 0.96. While the performance does not vary significantly when tweaking any of the tested classifier hyperparameters, variations can be observed when considering only the LLM parameters of the configurations.

The plot in Figure 9 shows the average F1-Scores of all configuration, after training and evaluating each classifier 40 times. Additionally, the same plot shows for each configuration whether it used the Alibaba embedding model with an overlap of 2048 ( $\frac{1}{2}$ ) or 512 ( $\frac{1}{8}$ ), or whether it used the bigger Qwen model with an overlap of 1024 ( $\frac{1}{2}$ ) or 256 ( $\frac{1}{8}$ ).

The plot reflects the findings yielded by the McNemar tests performed above. It seems like the Alibaba model with an overlap of  $\frac{1}{8}$  delivers the most useful embedding vectors for random forest classifiers.

It is difficult to determine an overall best configuration, as F1 performances do not vary strong enough. Nonetheless, to create a final detection model, a specific configuration must be fixed. This is done by determining the configuration with the highest average F1-Score.

This specific configuration yields an average F1-Score of 0.9478. It is trained on vectors embedded by the Alibaba model with an overlap of 512 Tokens. Furthermore, it uses a random forest classifier trained with 200 estimators, a maximum tree depth of 10, a minimum split of 5 and no bootstrapping. As this configuration is trained 40 times, a confidence interval based on the 2.5 and 97.5 percentile can be constructed:

$$F1_{95\% CI} = [0.9351, 0.9592]$$

While training the classifier on 40 different datasets, 40 different decision thresholds emerge from which the best value must be chosen for the final model. All thresholds range from 0.31 to 0.61. A more detailed report can be found in Appendix 4.

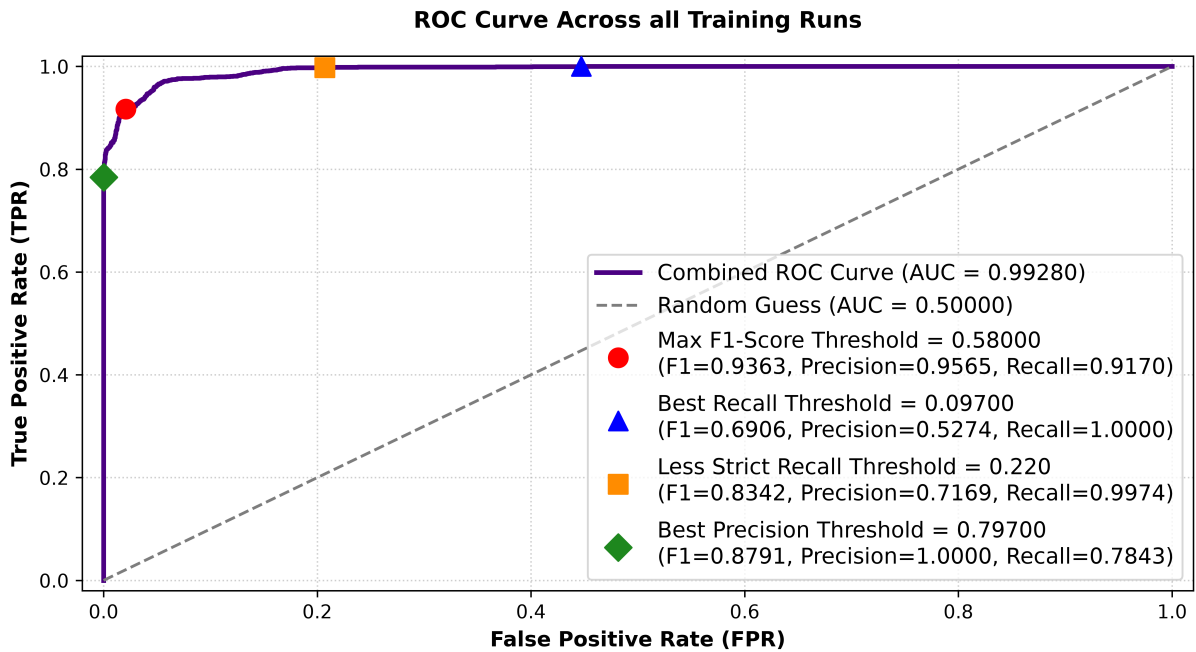


Figure 10: ROC curve of all predictions yielded during the training processes of the random forest classifier.

For binary classification tasks, a receiver operating characteristic (ROC) curve can help to tradeoff between sensitivity and specificity and determine the best decision threshold. A ROC curve plots the true positive rate against the false positive rate. Every combination of true positive rate and false positive rate has a corresponding decision threshold. In addition, the Area Under the ROC Curve (AUC) tells how well the model orders data, regardless of the classification threshold. An AUC of 0.99 shows that if one positive example and one negative example are randomly chosen, there is a 99% chance that the model assigns a higher score to the positive example than to the negative example. Therefore, an AUC of 1 would indicate perfect classification [12, 17].

The plot in Figure 10 shows an ROC curve when considering all predictions that the classifier made in its 40 training processes. The graph yields a high AUC of 99%, indicating that the classification model has a strong understanding of self-implemented crypto and non-crypto. The highest F1-Score is reached with a threshold of 0.58. Using this threshold, a true positive rate (Recall) of 0.92 and a false positive rate of 0.02 occur. The threshold can be optimized to hit a Precision of 1, while the false positive rate reaches near zero and the true positive rate drops to 0.78. On the other hand, the threshold can be optimized to hit a perfect Recall on the evaluation data, but this comes with a false positive rate of 0.45. For this reason, a less strict threshold, that still optimizes on Recall, is shown in the plot. It is set to the point where the gradient of the plot visibly starts to flatten. This threshold halves the false positive rate while minimally affecting the Recall.

#### 4.4.3 Summary

From the tests in Section 4.4.2, the best performing classifier uses the Alibaba-NLP LLM with a chunk size of 4096 Tokens and an overlap of 512 Tokens. The classifier is built upon a random forest classifier with 200 estimators, a maximum tree depth of 10, a minimum split of 5 and no bootstrapping.

The confidence interval for the F1-Score of this classifier ranges from 0.94 to 0.96. When optimizing the threshold for the best F1-Score, a decision threshold of 0.58 emerges. From now on, this threshold will be used when the classifier is compared to other models.

This shows that random forest classifiers work very well with classifying vector embeddings. It seems like no matter how the random forest hyperparameters are set the classifiers hit a dead end at an F1-Score of  $\approx 0.96$ . However, there might still be room for improvement when tweaking the embedding model which provides the vectors for the random forest classifiers.

| Optimized on F1-Score |      | Optimized on Precision |       | Optimized on Recall |        |
|-----------------------|------|------------------------|-------|---------------------|--------|
| F1                    | 94%  | Precision              | 100%  | Recall              | 99.74% |
| Threshold             | 0.58 | Threshold              | 0.797 | Threshold           | 0.220  |

Table 7: Thresholds optimized for different metrics.

The threshold can also be optimized for Precision. In this case, a decision threshold of 0.797 shows a Precision of 1 with a Recall of 0.78. In other words, this threshold yields no false positives among all 4200 negative examples (40 test runs \* [73 crypto-api sets +

32 crypto-unrelated sets], the same file occurs multiple times in the 4200 examples while some may be completely excluded). On the other hand, 22 of 100 files containing self-implemented crypto are not detected. This threshold can be used for applications where auditing false-positives is too expensive, for example in automated pipelines.

In contrast, a decision threshold of 0.097 yields a perfect Recall, where not a single positive among all 1.720 self-implemented crypto examples is overseen (40 test runs \* 43 self-implemented-crypto sets, the same file occurs multiple times while some may be completely excluded). However, this comes with a false positive rate of 0.45, which means that 45 of 100 files that are classified as crypto are classified as false positives.

However, the plot in Figure 10 also shows that a Recall of 99.74% can be achieved with a false positive rate of 19.69% at a decision threshold of 0.220. As this reduction in Recall does not significantly affect the classifier performance, optimizing the decision threshold for this Recall value essentially reduces the amount of false positives by more than half compared to the false positive rate of 45%, while only minimally increasing the risk for false negatives. Therefore, this threshold can be used for applications where as many self-implemented cryptographic functions as possible should be detected.

## 4.5 Building a Detection Model Based on a Fine-Tuned Large Language Model

The findings in Section 4.4.2 show that a random forest classifier is able to yield F1-Scores up to 0.96 when trained on vector embeddings provided by the Alibaba embedding model. The question remains if there is room for improvement. For example, the embedding model may not be able to clearly represent the difference of normal code to cryptographic algorithms within the vector space. This is because the embedding model is not trained specifically for cryptographic functions in code.

Thus, one improvement could be to fine-tune the embedding model to better separate cryptographic code from other code. Based on the embeddings provided by the fine-tuned embedding model, another random forest classifier could be trained, which may deliver better results.

Fine-tuning the Alibaba LLM utilizes a contrastive learning method that uses three different datasets:

- **anchors:** A dataset of chunks containing self-implemented cryptographic functions. These serve as reference points.



- **Positives:** Another dataset of chunks also containing self-implemented cryptographic functions. These are considered “similar” to the anchors.
- **Negatives:** A dataset of chunks from files that do not contain self-implemented cryptographic functions. These are considered “different” to the anchors.

The fine-tuning process trains the LLM to distinguish between similar and dissimilar code. For each anchor, the model learns to generate a vector embedding that is closer to the embeddings of positive samples and more distant from the embeddings of negative samples [50]. This reorganizes the vector space and pulls representations of cryptographic code together while pushing non-cryptographic code away, which improves the model’s ability to differentiate between the two categories.

#### 4.5.1 Identifying Cryptographic Chunks for Fine-Tuning

As explained earlier, there is no guarantee that every chunk within a file labeled as self-implemented crypto actually contains cryptography. Some chunks within these files may consist of noise only, especially when code in a file serves much more functionalities than only one implementation of a cryptographic algorithm. When fine-tuning the LLM, no chunks containing non-cryptographic code should be present in the anchors or positives dataset. Otherwise, this could significantly degrade performance, especially because the same chunk is used multiple times in the training process.

Hence, only chunks that are classified with high confidence as cryptographic code by the earlier random forest classifier will be included into the anchors and positives dataset. This is visualized as the fifth step “Use classifier to determine crypto chunks with high confidence” in Figure 11. To achieve this, the classifier must absolutely prevent to classify false positives, but can take the risk of yielding false negatives. For this, the classifier threshold for the strongest random forest classifier mentioned in Section 4.4.2 is optimized to detect not a single false positive. For reference, if this classifier uses a threshold of 0.779 (i. e., the classifier must be 77.9% confident that a file is crypto to actually classify it as crypto), not a single false positive occurs in all test runs, while an average of 9 false negatives per run happen.

However, this classifier uses all chunks of a file to decide whether to classify that file as crypto or not. But for the use case of fine-tuning the LLM, the decision should fall for a specific chunk, not the whole file. There is no guarantee that crypto unrelated chunks within a crypto file are actually classified as crypto unrelated. They might as well be classified as crypto, which would stay undetected, because they originate from a crypto

file. Therefore, to be extra sure no false positives occur in the anchor and positives dataset of the contrastive fine-tuning, the threshold for the classifier is set to 0.95.

To construct a dataset of positives, the random forest classifier is used to classify every chunk in the self-implemented crypto files from the **classifier’s training dataset**. Of course, no chunks in crypto files from the evaluation or validation dataset should be used to fine-tune the model, as this would manipulate the results later during evaluation and validation of the detection model. However, using crypto files from the training dataset of the classifier is not ideal either, as it is more likely, that a crypto unrelated chunk from a crypto file is classified as crypto since the classifier has learned to classify this exact chunk as crypto during its training process. Using chunks from this dataset works only under the assumption that the ensemble of trees is able to suppress the noise introduced by such chunks.

This could have been prevented, if a separate dataset of cryptographic code would have been set aside next to the training, validation and test datasets. However, this would reduce the amount of valuable data used for training the classifiers or testing. Finally, a classifier threshold of 0.95 classifies 368 of 509 chunks as containing self-implemented cryptographic functions. These chunks emerge from 154 of the 199 crypto files available in the training dataset.

#### 4.5.2 Pipeline of the Extended Detection Model

The workflow to create a detection model based on a fine-tuned LLM differs from the earlier workflow depicted in Figure 6 by appending a few additional steps:

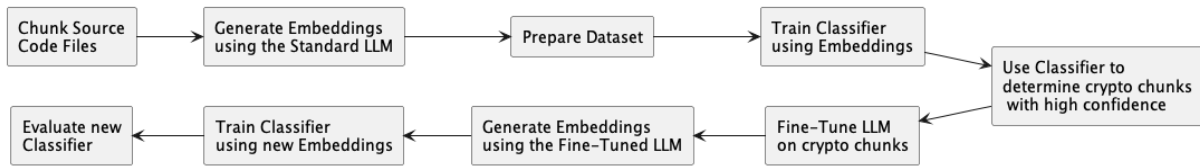


Figure 11: Pipeline of the improved detection model’s training process.

The first 4 Steps are already covered by the findings in Section 4.4.2. Hence, the standard LLM is the Alibaba embedding model and the classifier used is built on the random forest architecture with 200 estimators, a maximum tree depth of 10, a minimum split of 5 and no bootstrapping. As described above, this classifier is then used to determine 368 chunks that most likely ( $> 95\%$  confidence) contain self-implemented cryptographic code. These chunks originate from files labeled as containing self-implemented cryptographic code in the classifier’s training dataset.

In the next step, the anchors, positives and negatives datasets are constructed based on these crypto chunks and the chunks from files with crypto API usage or code unrelated to crypto. The datasets are constructed such that every anchor is assigned a positive, which is not the same chunk as the anchor itself, and the anchor is assigned a random negative. This results in 368 batches for fine-tuning the LLM. By applying the QLoRA method introduced in Section 2.6, the model learns to further differentiate self-implemented cryptographic code from other code. In order to enable fine-tuning on an NVIDIA 3060 Graphics Card with 12GB VRAM, the model is loaded with 4 bit quantization. For other fine-tuning parameters, like the LoRA target modules, standard values are used. The exact parameters can be found in the appendix.

After fine-tuning the LLM on the triplets, all chunks have to be embedded again, this time using the new fine-tuned embedding model. Like earlier, these embeddings are then used to train a classifier.

#### 4.5.3 Evaluating the Fine-Tuned Approach

Testing the extended detection model is a tedious process. Again, the model needs to be trained and tested on different training and validation datasets, such that a significant confidence interval can be determined. However, in this scenario, training and testing the model on several different datasets not only requires training several classifiers, but also fine-tuning the LLM and embedding the files based on the fine-tuned LLM. This makes detecting the best model configuration extremely difficult on the available hardware.

To be precise, the runs in Section 4.4.2 show that embedding a single file with the Alibaba-NLP model takes an average of about 3 seconds. For training and validation, 840 files must be embedded which therefore takes about 45 minutes. Considering that the fine-tuning process can also be very time-consuming, this suggests that a single training and validation process of the extended model takes longer than 45 minutes.

For this reason, only a single configuration is tested for the extended detection model. Both classifiers, the first classifier, from now on called **extended model**, (Step 8 in Figure 11) and the second classifier, from now on called **basic model**, (Step 4 in Figure 11) adopt the same parameters as the chosen classifier from Section 4.4.2, while the fine-tuning uses the parameters shown in Appendix 5.

Figure 12 shows the result of this experiment. In its 25 runs, the extended model is able to yield F1-Scores between 0.905 and 0.977, while the basic model shows 40 F1-Scores between 0.935 and 0.96.

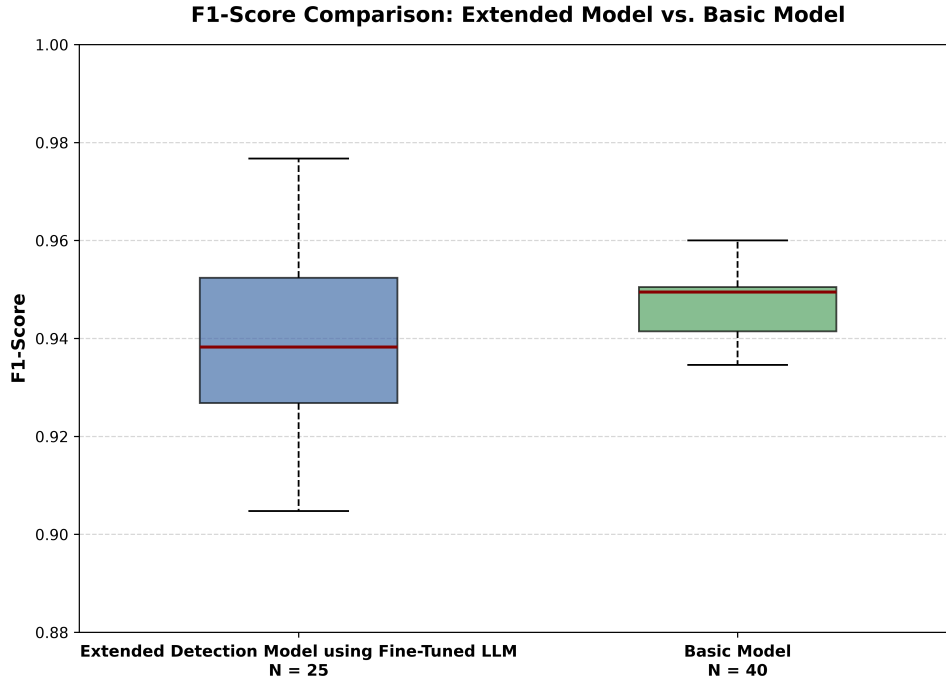


Figure 12: Comparison of the extended model performance with the performance of the basic classification model. The extended model is trained on 25 different datasets while the basic model is trained on 40 different datasets.

The extended model shows a much higher standard deviation ( $s_{\text{extended}} = 0.019$  vs.  $s_{\text{basic}} = 0.006$ ) and a lower median ( $m_{\text{extended}} = 0.938$  vs.  $m_{\text{basic}} = 0.949$ ) in its F1-Scores than the basic model. At first glance, it appears that embeddings provided by the fine-tuned LLM fail to outperform the embeddings from the same basic LLM. While this is true for the specific scenario explored in this work, the experiment shows that the basic model can be outperformed with fine-tuned embeddings. On only 25 different training and validation datasets, the extended model is able to score higher values than any of the 4000 test runs in Figure 9 can deliver.

This proves, that the extended model is able to overcome the 0.96 F1-Score limit observed in Section 4.4.2, which clearly highlights the potential of the fine-tuned approach. There are several issues that could explain why the extended model performs worse in average:

- Falsely classified chunks in the fine-tuning process: The LLM may be fine-tuned on chunks that do not contain any cryptographic code, but are flagged as such. An

attempt to overcome this problem was to use the basic model to classify all chunks and only use chunks that are classified as crypto with a confidence higher than 95%. However, this does not guarantee that no false-positive chunks occur and are used for the fine-tuning of the LLM. If the LLM learns to represent cryptocode and non-cryptocode more similarly in the vector space, the random forest may struggle to correctly classify them.

- **Unfortunate parameters for the random forest classifier:** Due to the complexity of the extended model's training process, only a single configuration of hyperparameters for the random forest is tested. However, the fine-tuning of the LLM causes the values of embedding vectors to change. One reason for the degrading performance could therefore be that the hyperparameters of the random forest need adaption to understand the new semantics hidden in the embedding vectors. Nonetheless, the extended model does not only perform worse than the basic model, but it also shows an exceptionally high variance in its outputs, which is unique among all models tested until now. Therefore, it is unlikely that this issue alone causes the extended model to perform worse.
- **Forgetting during fine-tuning:** In the fine-tuning process, the LLM learns to create different embedding vectors for text than before. This could cause the LLM to forget general code understanding capabilities, degrading overall embedding quality. Out of all possible reasons for degrading, this is the most difficult to mitigate.

Further studies should definitely consider the fine-tuning approach and test the extended model on different random forest hyperparameters. Also, the original dataset should be adapted to have chunk-wise labeling instead of providing labels on the file-level only.

## 5 Evaluation

### 5.1 Model Evaluation

As introduced in Section 4.1.3, the original dataset is split in training, validation and test dataset. Since Section 4 builds the model upon training and validation set only, the test dataset is not touched until now. This dataset can now be used to get the performance metrics of the final model. As the validation set is not needed anymore, it can be merged with the training dataset for training the classifier one last time.

| F1-Score Threshold |      | Precision Threshold |       | Recall Threshold |      |
|--------------------|------|---------------------|-------|------------------|------|
| Threshold          | 0.58 | Threshold           | 0.797 | Threshold        | 0.22 |
| F1-Score           | 84%  | Precision           | 97%   | Recall           | 93%  |
| TP                 | 34   | TP                  | 28    | TP               | 40   |
| TN                 | 101  | TN                  | 104   | TN               | 84   |
| FP                 | 4    | FP                  | 1     | FP               | 21   |
| FN                 | 9    | FN                  | 15    | FN               | 3    |

Table 8: Testing scores of the classifier on the test set for different threshold selection strategies.

When applying the Recall threshold, the model shows a Recall of 93% on the test dataset. As shown in Table 8, this means that 3 out of the 43 crypto files are misclassified as non-crypto while approximately every fifth non-crypto file is mistakenly flagged as self-implemented crypto. While this score is lower than the score yielded earlier in Section 4.4.3, this is expected behavior and a great result. The thresholds determined in the earlier chapter deliver better results, as they are optimized to fit the validation data. For classifying the test dataset, the threshold could also be optimized to hit a Recall of 100%, but the final model needs a fixed threshold upon which it makes its decisions on yet unseen data.

Table 9 shows how every cryptographic algorithm in the test set is classified with the final detection model and the Recall threshold. Among all algorithms, the two cryptographic functions SHA256 and HMAC-SHA256, which are reported as being implemented in systemd, are successfully classified. In a later section, the detection model will classify all C and C++ files in the systemd source code. This test will show

whether the detection model is able to detect the same algorithms in legitimate open source software.

| Cryptographic algorithm | Total Files in Test Set | TP        | FN       |
|-------------------------|-------------------------|-----------|----------|
| AES-GCM                 | 2                       | 2         | 0        |
| 3DES                    | 1                       | 1         | 0        |
| Blowfish                | 6                       | 6         | 0        |
| RC4                     | 5                       | 5         | 0        |
| ChaCha20-Poly1305       | 2                       | 2         | 0        |
| ECDSA                   | 3                       | 2         | 1        |
| Ed25519                 | 2                       | 2         | 0        |
| MD5                     | 3                       | 3         | 0        |
| SHA-1                   | 4                       | 4         | 0        |
| Poly1305                | 3                       | 3         | 0        |
| SHA-256                 | 1                       | 1         | 0        |
| SHA-512                 | 3                       | 3         | 0        |
| HMAC-SHA256             | 2                       | 2         | 0        |
| BLAKE2s                 | 4                       | 3         | 1        |
| X25519                  | 2                       | 1         | 1        |
| <b>Total</b>            | <b>43</b>               | <b>40</b> | <b>3</b> |

Table 9: Detection results at the selected Recall threshold.

## 5.2 Model Explainability

What’s left to analyze is how exactly the model interprets the source code to make its decision. To get clarity, answering the following questions can help to explain the detection model’s thinking:

1. Can the model detect self-implemented cryptographic functions solely on their mathematical algorithms, or does it rely on syntactical features, like function names and crypto-constants?
2. As the model is trained on many files containing uses of crypto-APIs, did it learn to reject any file that uses such APIs, even if it implements its own cryptographic function?

For this, existing files can be permuted and reclassified. Concretely, 4 crypto files are manually modified: a SHA256, an MD5, a ChaCha and an RC4 implementation. The

files are deliberately chosen to cover a large range of classification scores in order to get a good picture about the effect permutations can have. While the SHA256 and RC4 file would fail to be identified as crypto with the F1-Score threshold, the remaining two are successfully classified as crypto. For all files, three different versions are created:

- **Comments only:** Every piece of code except for comments are removed.
- **Generalization:** Every user-introduced syntactic context is removed or replaced with generics. This includes comments and variable and function names. In addition, all numerical constants are replaced with generic variable names.
- **API-Obfuscation:** At the top and bottom of the crypto files, 2 functions are added which use well-known crypto libraries' APIs. These functions are scraped from files in the validation dataset that use crypto-APIs.

This leaves a total of 12 files. Ideally, nine of them are classified as crypto and the remaining three, the ones where only the comments remain, are assumed non-crypto.

|                 |        |        | Initial Classification Scores and Diffs |        |        |        |
|-----------------|--------|--------|-----------------------------------------|--------|--------|--------|
| Category        | TP/TN  | FP/FN  | SHA256                                  | RC4    | ChaCha | MD5    |
| Original        | TP = 2 | FN = 2 | 0.274                                   | 0.562  | 0.917  | 0.88   |
| Comments only   | TN = 1 | FP = 3 | -0.040                                  | +0.110 | -0.170 | -0.056 |
| Generalization  | TP = 4 | FN = 0 | +0.621                                  | +0.115 | -0.047 | -0.127 |
| API Obfuscation | TP = 2 | FN = 2 | +0.046                                  | -0.076 | -0.259 | -0.260 |

Table 10: Classification results for 12 permuted files using an F1-score threshold of 0.58.

The results show that comments can have a significant effect on code classifications. As shown in Table 10, scores stay relatively stable when only considering source code comments. All permuted files consist of detailed explanations of the underlying source code in natural language. In addition, removing natural language context like comments and variable names seems to cause higher classification scores. Especially the generalized SHA256 code shows a strong improvement in classification compared to the original file.

This strongly indicates that natural language in source code can be a strong noise factor. However, this also shows that the detection model is indeed able to detect cryptographic code solely on the abstracted form of cryptographic algorithms. This is a very important insight that future work on this topic should definitely catch on. Automated generalization techniques could preprocess code files before classifying with the potential for higher success rates.

Lastly, natural language in source code is not the only factor that might obfuscate code and prevent successful detection. Apparently, obfuscating self-implemented cryp-



tographic functions with uses of unrelated crypto-APIs worsens classification scores. Therefore, it can be expected that self-implemented cryptographic code that additionally provides functionality based on crypto-API uses might have a higher risk of being mistakenly labeled as non-crypto.

Of course, permutating 4 files is not enough to provide statistical evidence for these hypotheses. For this, more files would need to undergo the permutation and classification process. However, the findings give suggestions for the direction of future work in this manner.

### 5.3 Detecting Self-Implemented Cryptographic Code in Systemd

To get a general idea of how well the detection model performs in a real-life scenario, the final model is assigned the task to analyze the systemd source code for possible self-implementations of cryptographic functions. The Introduction in Section 1 mentioned that the systemd project ships with SHA256 and HMAC-SHA256 code files that do not use any standard cryptographic API. Therefore, the model can detect at least these two files and may even find additional self-implementations. In case other cases are detected, this would be valuable information for Garden Linux, as the project depends on systemd.

As the detection model is trained on C and C++ code files, only those programming languages are considered within the systemd codebase. There are around 4900 files in the systemd repository, 1855 of them are C code. Systemd does not use any C++ code.

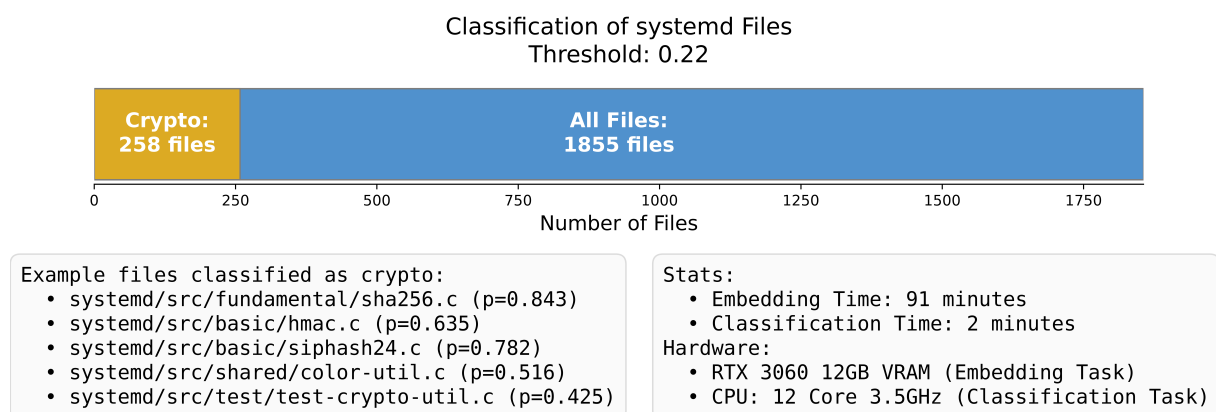


Figure 13: Stats and Classification Results of all C files in systemd

Out of the 1855 C files in systemd, 258 (14%) are classified as crypto at the selected Recall threshold. Among these files, the SHA256 and HMAC-SHA256 implementations mentioned in this work’s introduction are successfully classified as cryptographic code. The model also identifies other self-implemented hashing algorithms, such as SipHash. This hash function is a good example for code that may be relevant for FIPS certification. SipHash is designed to be resistant against flooding attacks while being simple to calculate. This resistance comes from the ability to integrate a secret key into the hashing algorithm, which makes it much harder for attackers to deliberately cause hash collisions [51].

The self-implemented SipHash function has been introduced to systemd in 2013 with the goal of standardizing the use of hashing algorithms in systemd [52, 53]. Systemd uses the cryptographic feature of SipHash as proven by a commit, where this hash function has been implemented into systemd’s `journal` feature in order to prevent collision attacks [54]. If wrongly implemented, attackers may be able to flood this service with inputs that all result in the same hash, which could then result in a denial of service.

Another file that is classified as crypto is a color utility file that provides a lightweight functionality to convert RGB values in HSV and vice-versa. As this file contains a lot of mathematical calculations, this file is a good candidate for an additional math category as mentioned in Section 4.1.3. If the detection model would have learned to better distinguish crypto-unrelated math and cryptographic algorithms, this false positive may have been prevented.

Apart from that, the model classifies some test files as cryptographic. Most likely, those files do not need any auditing as they provide no functionality. Instead of embedding all files in the codebase, a list of relevant or irrelevant files could be defined, such that only code of interest is considered.

Inspired by the work of Dozono et al., the model workflow can be extended by classifying all 258 crypto-classified files using strong chat LLMs. As their study shows that classification performance improves when taking larger and more capable models [37], the “Gemini 2.5 Pro” model is used to analyze the 258 files. For each file, the chat LLM is prompted the following:

You are an AI classifier that identifies whether the provided code contains self-implemented cryptographic code. You should respond with either ‘contains self-

implemented cryptographic code’ or ‘does not contain self-implemented cryptographic code’. Explain your decision in a short sentence.

This results in a total of 16 files that the Gemini model claims to contain self-implemented crypto code, while the remaining 242 files are flagged as false positives. While not all suggestions are manually reviewed, it is remarked that the LLM can make mistakes in classifying files.

Systemd delivers a file called “lookup3.c” that provides hash functions for fast hash table lookup. This thesis’ model classifies this file as “contains self-implemented cryptography” with a probability of  $p = 0.874$ , which is the second highest probability score yielded for all files in the systemd codebase. While this file does contain a self-implemented hash function, the Gemini model sorts it into the non-crypto class with the following explanation:

does not contain self-implemented cryptographic code

This file implements Bob Jenkins’ public-domain lookup3 non-cryptographic hash (hashlittle/hashword/mix/final) intended for hash-table lookup and explicitly states it is not suitable for cryptographic purposes.

One could argue, that the applied prompt may not provide enough details for the model to fully understand its task, but even if the instruction were modified to include details about FIPS importance, the model is still vulnerable to misleading language. For example, the model should still classify a crypto file as such even if it contains a comment that states something like “this function is used for non-cryptographic purposes only, which makes it irrelevant for FIPS certification.”.

This is a different research topic, which is out of this scope for this thesis (e. g. [55], [56]).

## 5.4 Comparison with Related Work

In Section 3.3, several projects related to the topic of detecting cryptography in software are introduced. However, only few projects allow to identify self-implemented cryptography. This includes the FoC tool [35], Lestringant’s work [32], the grap-tool [30] and Aligot [31]. Among the 4 tools, only the FoC tool can realistically be used for comparison with the model introduced in this thesis due to its architectural similarity and because sufficient test metrics are provided.

FoC deploys LLMs to detect cryptographic functions in compiled binary files. This differs from the way the tool introduced in this thesis analyzes software, which makes comparing both models not straightforward. While FoC detects cryptography in binaries on function level, this thesis classifies source code on file level. In their paper on FoC, the authors discuss 45 different cryptographic primitives which they used to train and evaluate their model on. According to their manual analysis, a total of 70 instances of primitive classes appear in the `x86_64` compiled versions of Libsodium, GmSSL, tiny-AES-c and wolfSSL. The authors claim, that their model is able to identify 46 out of these 70 primitive classes [32].

To measure how good this thesis' work compares with FoC, the source code of all C and C++ files in the four libraries is analyzed with the detection model. As this is about comparing the performance of two models, the F1-Score threshold of 0.58 is applied. This yields a total of 172 files that are classified as crypto. Manual analysis reveals that 137 files are related to the implementation of one of the 70 primitives. This is because the implementation of a cryptographic algorithm can be spread across multiple source code files.

To conclude, within the source code of the 4 libraries, the detection tool is able to detect **45** out of the 70 primitives mentioned above, which is one less than the FoC-tool claims to identify. However, it is important to consider the different class distributions that both models are trained on. While the FoC-tool's training data consists of all 45 cryptographic algorithms that are later used for testing, this thesis' detection model is trained on only 18 of these 45 algorithms. A detailed comparison can be found in Appendix 6. In fact, 74% of all correctly identified cryptographic algorithms were also used in training of the detection model. Furthermore, it is unclear whether all implementations that are searched for are written in C or C++, which could explain why they are missed by this work's model. Overall, this indicates that a broader training set could have improved model performance in this scenario.

Additionally, while the authors of FoC report that analysis of the four libraries takes only a few seconds, the detection model introduced in this thesis runs for more than one hour to embed and classify all source code files. On the one hand, this difference can be explained with the fact that FoC uses a lighter LLM for detection. On the other hand, the authors report to have used a hardware setup consisting of 10 high-end GPUs and a 48-core CPU [32], which is significantly stronger than the simple consumer hardware used in this thesis.

## 5.5 Economic Aspects

There is no simple calculation for comparing the costs of achieving FIPS compliance with and without the tool developed in this thesis. The costs associated with FIPS validation depend on factors such as the number and type of cryptographic modules, documentation requirements, and fees for certified testing laboratories. On the other hand, failing to identify cryptographic modules that require validation can introduce compliance and financial risks, particularly where the use of non-validated cryptography violates rules and contracts.

The path to FIPS compliance can be very expensive, as a lot of workload is spent on manually auditing code files. The detection tool introduced in this thesis significantly reduces the amount of time spent to manually detect files using cryptographic algorithms in source code to a fraction of the original workload. For example, when classifying all C files in systemd, the model suggests auditors to focus on about 14% of all files.

In a discussion within the Garden Linux team, the team manager decided to set an auditing budget of 10.000 lines of code per day to identify FIPS relevant code in the Garden Linux codebase. With this budget, the systemd project can be analyzed in around 3 days (28.000 lines of code) when only auditing the files that are flagged as potentially crypto. This is much cheaper than analyzing the whole codebase for 21 days (204.000 lines of code).

However, the integration of the tool does not come without any costs, as files need to be embedded with an LLM on strong hardware. Most developers in the Garden Linux team use standard working Laptops that may come with strong CPUs but weaker GPUs that are sufficient for normal everyday use. A small experiment on an M2 Pro Chip by Apple shows that embedding a single chunk takes about 10 seconds. Compared to the embedding times measured earlier, it can be assumed that running the detection tool on this hardware takes about 10 times longer.

Consequently, using the detection model requires either spending on better hardware or long computing times. However, even when temporarily investing in servers with the required hardware, it is unlikely that this spending would reach anywhere near the cost of manual auditing.

Another economical aspect lies in the use of chat LLMs like Claude Sonnet or Gemini Pro as assistance in the labeling process for the dataset or as an assistant to auditors.

Assuming that the LLM is prompted one instruction for every file, the following token usages emerge:

| Model             | Input Tokens | Output Tokens | Sample Size         |
|-------------------|--------------|---------------|---------------------|
| Claude Sonnet 4.6 | 264          | 2.645         | Average of 22 files |
| Gemini 2.5 Pro    | 1.282        | 81.527        | Average of 10 files |

Table 11: Average token usages when prompting the Claude Sonnet 4.6 model or the Gemini Pro 2.5 model to classify a file using the prompt described in Section 5.3. For both tools, the respecting official command line tool is used.

While this thesis used the “Claude Sonnet 4.6” model for assisting the file labeling (Section 4.1.3), the “Gemini 2.5 Pro” model is used for assisting the auditing process for the systemd project (Section 5.3). Following the official pricing information for both models, the labeling of all 988 files costs around 39.98\$ [57] while the classification of the 258 files in systemd costs around 210.75\$ [58].

Note that this calculation uses the official pricing model instead of the actual pricing agreement between SAP and both companies. Also, token caching is not considered into the calculation. While this results in slightly fewer actual costs, the numbers still show that the costs should be considered when choosing a LLM as an assistant. For example, in the experiments conducted in this thesis, the “Claude Sonnet 4.6” model shows good classification capabilities while being cheaper than the “Gemini 2.5 Pro” model.

## 5.6 Sustainability

The development and application of the cryptographic detection model has several implications for sustainability. Firstly, the computational resources required for embedding files with LLMs and training the detection model are significant. While there are benefits in form of reduced manual auditing costs, the energy consumption that comes with the usage of powerful hardware contributes to a carbon footprint.

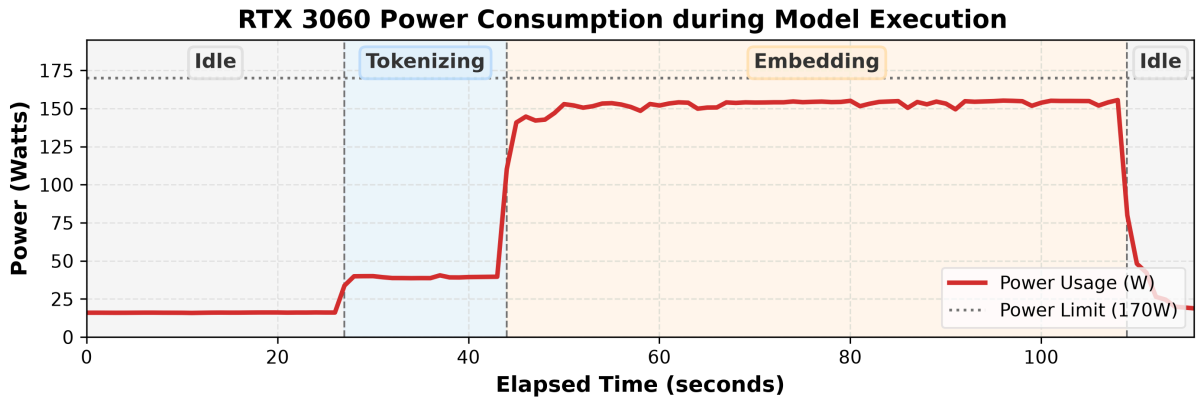


Figure 14: Power consumption of the GPU when running the detection model on systemd. Process is interrupted after 110 seconds.

As shown in Figure 14, the graphics card consumes about 150 Watts when embedding files. When running for 90 minutes, as was the case for detecting cryptography in systemd, this adds up to a total of 225 Wh or 0.12 Wh per file. Thus, before using the model, a list of files that should be excluded from analysis (like test files) should be defined to save time and energy.

Secondly, investing in high-performance computing infrastructure can be costly in monetary terms and potentially in environmental impact. However, the alternative of long computing times on less powerful hardware also implies a longer period of energy consumption. Its possible that the use of cloud-based, on-demand GPU resources that scale computing power only when needed, can be more energy efficient than laptop hardware.

In addition, the detection models maintainability must be considered. As cryptographic methods evolve and standards change, the model will require updates and retraining. This process too consumes energy.

However, it is important to note that in production, this model won't analyze whole codebases continuously. At first, it will be used to analyze all relevant open source projects. Later, it runs as part of a pipeline where potentially relevant code changes are automatically analyzed.

Finally, the usage of assisting LLMs like "Claude Sonnet" can have an additional environmental impact. While actual numbers often remain confidential, scientists try to estimate the environmental impact of LLMs using accessible data. For example, the work by Nidhal Jegham et al. estimates energy consumption, carbon footprint and water usage of state-of-the-art LLMs using official numbers and API performance metrics.

They estimate energy consumption of the “Claude Sonnet 3.7” model to be at around 2.8 Wh per 1000 input and output tokens [59].

Assuming the labeling in Section 4.1.3 were assisted by this model (instead of “Claude Sonnet 4.6”), the environmental impact could be estimated to around 28.000 Wh energy used, 1kg CO2 emitted and 9.5l consumed water (Assuming 1000 input and 1000 output tokens consumed for each of the 988 files). It can be assumed that resource consumptions of the 4.6 model and the 4.7 model are comparable. While such estimations have their limitations and shouldn’t be taken for granted, they show that LLM assistance must be well thought-out before being used. For example, the token usages discussed in Section 5.5 show that the “Gemini 2.5 Pro” model consumed a lot more tokens than the “Claude Sonnet 4.6” LLM, which does not only result in much higher pricing but also stronger environmental impact. More lightweight models may achieve equally good performance while having fewer negative effects on the environment.



## 6 Conclusion

This thesis investigated the automated detection of self-implemented cryptographic functions in C and C++ source code with the goal of supporting Garden Linux on its path to FIPS 140-3 compliance. The need for this tool comes from the observation that some open-source projects contain self-implemented or copied cryptographic implementations which cannot inherit the assurance of a validated library and therefore require additional certification. The work presented showed a vector-embedding based pipeline that evaluated multiple classifier architectures and embedding strategies. Finally, it explored a fine-tuning approach to improve detection quality.

### 6.1 Summary of Findings

The core method slices source code files into chunks, generates high-dimensional embeddings with LLM, and classifies those embeddings using binary classifiers. Experiments tested 4,860 classifier configurations across three embedding models and a variety of classifier hyperparameters. The results are:

- A random-forest classifier trained on embeddings from the Alibaba-NLP model achieved the best average performance. The strongest configuration (Alibaba embeddings, 512 token overlap, RandomForest with 200 estimators, max depth 10, min split 5) yielded a strong Recall of 93% and a 97% Precision (depending on the threshold used).
- Threshold selection strongly affects how the model is used in practice: a threshold of 0.58 optimizes F1 in order to enable comparison, 0.797 yields near-perfect Precision, and 0.22 maximizes Recall. The flexible choice of thresholds makes the detection model highly adaptable to specific use cases.
- In a real-world case study, the final detector analyzed 1,855 C files of systemd and flagged 258 files ( 14%) as potentially containing self-implemented cryptographic code. The known SHA-256 and HMAC-SHA256 implementations in systemd were successfully identified, and other candidates such as SipHash were detected as well.
- Fine-tuning the embedding model (training on high-confidence crypto chunks) demonstrated potential to break the apparent F1 ceiling of 0.96 but produced higher variance and mixed average performance in the experiments. This indicates that there is potential, but also the need for further methodological investigations (e. g., cleaner chunk-level labels and prevention of forgetting).

- Chat LLM-assisted labeling can help with annotation and assist auditing, but carry monetary and environmental costs. Also Chat models used can produce incorrect classifications and misleading explanations. While this couldn't be observed in the LLM-assisted labeled dataset used to train this thesis' model, at least one misclassification occurred when using chat LLMs to validate the crypto-classified files in systemd.

## 6.2 Contributions

This thesis makes the following contributions:

- A detection pipeline based on code embedding models with classical classifiers that is directly applicable to codebases of open source projects used by Garden Linux.
- An evaluation on both labeled datasets and a real-world codebase (systemd), demonstrating the tool's practicability for auditors and maintainers.
- An example of a PEFT fine-tuning workflow (QLoRA) to fine-tune embeddings to better understand cryptographic code.

The workflow for reproducing the training and using this model can be found on the following GitHub page: <https://github.com/Sagichnicht04/CryptoVectorClassification>

## 6.3 Practicalities for Garden Linux and FIPS Certification

The detector provides a good step toward reducing manual audit effort required for FIPS certification. In practice, the model can be configured in two ways:

- Audit mode (high Recall): Use a low decision threshold to prioritize recall. This maximizes the number of detected crypto files, ensuring fewer missed self-implementations at the cost of more manual reviews.
- Pipeline mode (high precision): Use a strict threshold that minimizes false positives. This is suitable for automated detection or brief manual review.

For Garden Linux, the recommendations are straightforward: The detector should run over new or changed code in CI, present detected files to human auditors with contextual information, and maintain an exclusion list (tests, unrelated utilities) to reduce runtime and energy consumption. Even with modest hardware, the detector reduces the set of possible files significantly (for systemd the tool suggested auditors to focus on roughly 14%).

## 6.4 Limitations

The approach presented in this thesis has several important limitations that must be considered when applying in production:

- **Data gathering based on known cryptographic identifiers:** The dataset is constructed by crawling repositories for known cryptographic identifiers. The model is never trained or tested on self-invented or unconventional cryptographic functions such that they may remain undetected. Nonetheless, it successfully identified cryptographic primitives in cryptolibraries that it has not been trained on (Section 5.4).
- **Chat LLM-assisted annotation introduces potential for human errors:** The labeling process relies on an LLM’s suggestions, with human oversight. If the human annotator’s knowledge is insufficient, this could lead to inaccuracies in the dataset labeling. An extensive manual analysis of 35 files in the dataset has been conducted where no mislabeled file could be identified, which reduces the realistic highest mislabeling rate to 8.6%. But an experiment where a chat LLM was instructed to validate the crypto-classified files in systemd showed that LLM-assistance can make mistakes and is vulnerable to misleading language.
- **File-level labeling leading to incorrect chunk labels:** The dataset labeling is performed at the file level (“contains self-implemented crypto” or “does not contain self-implemented crypto”). However, the model processes code in smaller chunks. This can result in chunks being incorrectly labeled during training if a file contains both cryptographic and non-cryptographic code, which potentially harms model performance.
- **Source Code only:** The detection model is trained on uncompiled source code files and is not intended to detect cryptographic functions within compiled binaries. Related work, like the earlier introduced “FoC” by Shang et al., shows that LLMs are also able to process and understand pseudocode yielded by reversed-engineered binary files.
- **C and C++ only:** The detection model learned its capabilities from C and C++ code only and has never seen any cryptographic examples from other programming languages. However, this does not necessarily mean that the model doesn’t work with other programming languages. As seen earlier, when testing the model on files where code is removed and only comments remain, the model is still able to detect cryptographic usages. Still, performance on other programming languages would likely increase if used in training.

- **False positives:** The model occurs to classify non-relevant code, such as mathematical utility files and test files as cryptographic. To mitigate this, a list of files to be excluded from analysis should be defined and the model should be trained on more data from different categories, like mathematical algorithms.
- **Unmodified code representation:** The model learns and is evaluated on source code files as is without any modifications. Most related work mentioned in Section 3.3 run some kind of preprocessing before working with the code. This also applies for studies where LLMs are used. Generalizing the code can prevent syntactic noise and obfuscation and therefore improve overall model quality.
- **Explainability:** The model provides detection scores at chunk level but limited human-readable explanations for why a file was flagged.

## 6.5 Future Work

While the experiments in this work yield promising results, several improvements to detection quality and broader applicability could be made.

- **Dataset and labeling:** Move from file-level to chunk-level labels and include more examples (including negative examples like math, compression or image-processing code). This can reduce label noise and false positives.
- **Annotation quality:** Identify potential mislabeled files as a result of leveraging the “Claude Sonnet 4.6” model to assist file labeling or further reduce the highest realistic mislabeling rate by confirming more file labels.
- **Preprocessing and generalization:** Apply code generalization (strip comments, normalize identifiers, fold constants) to reduce syntactic noise and runtime and implement exclusion lists for irrelevant files (e. g., exclusion lists or path-based filtering).
- **Model robustness and training:** Investigate multi level models that combine graph features (e. g., DFGs) with embedding-based classifiers. This may improve robustness against syntactic obfuscation and API-noise. Also, research on how to mitigate forgetting during QLoRA fine-tuning and change the hyperparameters for random forest parameters after fine-tuning since embedding statistics change.
- **Language and binary coverage:** Extend the dataset to additional languages (Rust, Go, Python) and experiment with decompiler output to support detection in binary files as conducted by other related work.

- **Efficiency and CI integration:** Evaluate smaller embedding models, consider on-demand cloud GPUs if possible, and add a CI job that embeds only changed files and produces a prioritized reviewer list.
- **Explainability and auditor tools:** Add precise, human-readable explanations (like short summaries).
- **Operational adoption:** Define a Garden Linux workflow for handling flagged files.

These directions prioritize reducing label noise, improving preprocessing, strengthening model robustness, widening coverage, and adding explainability. These steps would shorten manual review time and make the model practical for Garden Linux.

## 6.6 Final Remarks

Detecting self-implemented cryptographic functions in large, diverse codebases is a challenging problem. The pipeline developed in this thesis shows that embedding techniques with LLM combined with binary classifiers can reduce manual audit effort while delivering high Recall when required. Future work that focuses on cleaner chunk-level labels, smarter preprocessing and explainability will increase detection performance.

## Bibliography

- [1] R. S. Geiger *et al.*, “The Labor of Maintaining and Scaling Free and Open-Source Software Projects,” *Proceedings of the ACM on Human-Computer Interaction*, vol. 5, no. CSCW1, pp. 1–28, Apr. 2021, doi: [10.1145/3449249](https://doi.org/10.1145/3449249).
- [2] BlackDuck, “2026 Open Source Security and Risk Analysis Report,” technical report, 2026. Available: <https://github.com/weltcomm/annual-security-reports/blob/main/Markdown%20Conversions/2026/BlackDuck-Open-Source-Risk-Analysis-Report-2026.md#obsolescence-management-and-maintenance-debt>
- [3] S. Inc., “10th Annual State of the Software Supply Chain.” Accessed: Aug. 26, 2026. [Online]. Available: <https://www.sonatype.com/state-of-the-software-supply-chain/2024/introduction>
- [4] *Security requirements for cryptographic modules*. 2019. doi: [10.6028/nist.fips.140-3](https://doi.org/10.6028/nist.fips.140-3).
- [5] S. Contributors, “libsystemd/sd-id128: use only internal hmac, remove khash/OpenSSL support .” Accessed: July 08, 2026. [Online]. Available: <https://github.com/systemd/systemd/pull/20915>
- [6] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: [10.1023/a:1010933404324](https://doi.org/10.1023/a:1010933404324).
- [7] scikit-learn developers, “RandomForestClassifier .” Accessed: July 23, 2026. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- [8] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [9] T. P. Community, “BCEWithLogitsLoss.” Accessed: July 14, 2026. [Online]. Available: <https://docs.pytorch.org/docs/2.13/generated/torch.nn.BCEWithLogitsLoss.html>
- [10] T. P. Community, “Neural Networks.” Accessed: July 14, 2026. [Online]. Available: [https://docs.pytorch.org/tutorials/beginner/blitz/neural\\_networks\\_tutorial.html#neural-networks](https://docs.pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial.html#neural-networks)
- [11] I. Goodfellow *et al.*, *Deep Learning*. MIT Press, 2016.
- [12] C. D. Manning *et al.*, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [13] S. Raschka, “Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning.” arXiv, 2018. doi: [10.48550/ARXIV.1811.12808](https://doi.org/10.48550/ARXIV.1811.12808).

- [14] R. Kohavi, “A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection,” 1995.
- [15] J.-H. Kim, “Estimating classification error rate: Repeated cross-validation, repeated hold-out and bootstrap,” *Computational Statistics & Data Analysis*, vol. 53, no. 11, pp. 3735–3745, 2009, doi: [10.1016/j.csda.2009.04.009](https://doi.org/10.1016/j.csda.2009.04.009).
- [16] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. 1994.
- [17] T. Hastie *et al.*, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, Second. Springer Science & Business Media, 2009.
- [18] T. G. Dietterich, “Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms,” *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998, doi: [10.1162/089976698300017197](https://doi.org/10.1162/089976698300017197).
- [19] T. Mikolov *et al.*, “Efficient Estimation of Word Representations in Vector Space.” arXiv, 2013. doi: [10.48550/ARXIV.1301.3781](https://doi.org/10.48550/ARXIV.1301.3781).
- [20] D. Operationnelle *et al.*, “A Neural Probabilistic Language Model,” Oct. 2001.
- [21] T. Kudo and J. Richardson, “SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, E. Blanco and W. Lu, Eds., Brussels, Belgium: Association for Computational Linguistics, Nov. 2018, pp. 66–71. doi: [10.18653/v1/D18-2012](https://doi.org/10.18653/v1/D18-2012).
- [22] Z. Feng *et al.*, “CodeBERT: A Pre-Trained Model for Programming and Natural Languages.” arXiv, 2020. doi: [10.48550/ARXIV.2002.08155](https://doi.org/10.48550/ARXIV.2002.08155).
- [23] E. Frantar *et al.*, “GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.” [Online]. Available: <https://arxiv.org/abs/2210.17323>
- [24] T. Dettmers *et al.*, “QLoRA: Efficient Finetuning of Quantized LLMs.” [Online]. Available: <https://arxiv.org/abs/2305.14314>
- [25] J. Devlin *et al.*, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds., Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, pp. 4171–4186. doi: [10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423).

- [26] A. K. Biswas *et al.*, “Training-free layer selection for partial fine-tuning of language models,” *Information Sciences*, vol. 740, p. 123204, 2026, doi: <https://doi.org/10.1016/j.ins.2026.123204>.
- [27] S. SE, “Garden Linux Docs.” Accessed: June 11, 2026. [Online]. Available: <https://github.com/gardenlinux/gardenlinux/tree/main/docs>
- [28] W. Li *et al.*, “CryptoGo: Automatic Detection of Go Cryptographic API Misuses,” in *Proceedings of the 38th Annual Computer Security Applications Conference*, in ACSAC. ACM, Dec. 2022, pp. 318–331. doi: [10.1145/3564625.3567989](https://doi.org/10.1145/3564625.3567989).
- [29] S. Rahaman *et al.*, *CryptoGuard: High Precision Detection of Cryptographic Vulnerabilities in Massive-sized Java Projects*. in ACM Digital Library. New York, NY, United States: Association for Computing Machinery, 2019.
- [30] L. Benedettia *et al.*, “Detection of cryptographic algorithms with grap,” 2017, Available: <https://eprint.iacr.org/2017/1119.pdf>
- [31] J. Calvet *et al.*, “Aligot: cryptographic function identification in obfuscated binary programs,” in *Proceedings of the 2012 ACM conference on Computer and communications security*, in CCS’12. ACM, Oct. 2012, pp. 169–182. doi: [10.1145/2382196.2382217](https://doi.org/10.1145/2382196.2382217).
- [32] P. Lestrinant, “Identification of cryptographic algorithms in binary programs,” *HAL theses*, 2018, Available: <https://theses.hal.science/tel-01699544>
- [33] H. Baek *et al.*, “CryptoLLM: Harnessing the Power of LLMs to Detect Cryptographic API Misuse,” in *Computer Security – ESORICS 2024*, Springer Nature Switzerland, 2024, pp. 353–373. doi: [10.1007/978-3-031-70879-4\\_18](https://doi.org/10.1007/978-3-031-70879-4_18).
- [34] Z. Li *et al.*, “CryptoScope: Utilizing Large Language Models for Automated Cryptographic Logic Vulnerability Detection.” arXiv, 2025. doi: [10.48550/ARXIV.2508.11599](https://doi.org/10.48550/ARXIV.2508.11599).
- [35] X. Shang *et al.*, “FoC: Figure Out the Cryptographic Functions in Stripped Binaries with LLMs,” *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 1, pp. 1–38, Dec. 2025, doi: [10.1145/3731449](https://doi.org/10.1145/3731449).
- [36] R. Shree *et al.*, “Automated Recognition of Cryptographic Algorithms via Machine Learning,” in *Evolutionary Artificial Intelligence*, Springer Nature Singapore, 2025, pp. 171–181. doi: [10.1007/978-981-96-5210-5\\_13](https://doi.org/10.1007/978-981-96-5210-5_13).
- [37] K. Dozono *et al.*, “Large Language Models for Secure Code Assessment: A Multi-Language Empirical Study.” [Online]. Available: <https://arxiv.org/abs/2408.06428>



- [38] N. Liu *et al.*, “Do LLMs Make Mistakes Like Students? Exploring Natural Alignment between Language Models and Human Error Patterns.” [Online]. Available: <https://arxiv.org/abs/2502.15140>
- [39] B. D. Jovanovic and P. S. Levy, “A Look at the Rule of Three,” *The American Statistician*, vol. 51, no. 2, pp. 137–139, 1997, doi: 10.1080/00031305.1997.10473947.
- [40] T. Sanders, “Classification using embeddings .” Accessed: June 29, 2026. [Online]. Available: [https://developers.openai.com/cookbook/examples/classification\\_using\\_embeddings](https://developers.openai.com/cookbook/examples/classification_using_embeddings)
- [41] Huggingface, “CodeBERT-base .” Accessed: July 23, 2026. [Online]. Available: <https://huggingface.co/microsoft/codebert-base>
- [42] Huggingface, “Alibaba-NLP/gte-Qwen2-1.5B-instruct.” Accessed: Aug. 03, 2026. [Online]. Available: <https://huggingface.co/Alibaba-NLP/gte-Qwen2-1.5B-instruct>
- [43] Huggingface, “Qwen2.5-7B-Instruct .” Accessed: July 23, 2026. [Online]. Available: <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>
- [44] A. C. Short and A. Z. Henley, “Towards an Empirically-Based IDE: An Analysis of Code Size and Screen Space,” 2019. doi: 10.1109/VLHCC.2019.8818857.
- [45] J. L. Ba *et al.*, “Layer Normalization.” arXiv, 2016. doi: 10.48550/ARXIV.1607.06450.
- [46] T. P. Community, “LayerNorm .” Accessed: Aug. 04, 2026. [Online]. Available: <https://docs.pytorch.org/docs/main/generated/torch.nn.modules.normalization.LayerNorm.html>
- [47] T. P. Community, “GELU .” Accessed: Aug. 04, 2026. [Online]. Available: <https://docs.pytorch.org/docs/2.13/generated/torch.nn.GELU.html>
- [48] T. P. Community, “Dropout .” Accessed: Aug. 04, 2026. [Online]. Available: <https://docs.pytorch.org/docs/main/generated/torch.nn.modules.dropout.Dropout.html>
- [49] G. E. Hinton *et al.*, “Improving neural networks by preventing co-adaptation of feature detectors.” arXiv, 2012. doi: 10.48550/ARXIV.1207.0580.
- [50] F. Schroff *et al.*, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.

- [51] J.-P. Aumasson and D. J. Bernstein, “SipHash: a fast short-input PRF.” [Online]. Available: <https://eprint.iacr.org/2012/351>
- [52] L. Poettering, “Commit 9bf3b53 - shared: switch our hash table implementation over to SipHash .” Accessed: Aug. 13, 2026. [Online]. Available: <https://github.com/systemd/systemd/commit/9bf3b53533cdc9b95c921b71da755401f223f765>
- [53] L. Poettering, “Commit b67f541 - bus: switch kdbus bloom filter over to SipHash (from MurmurHash3) .” Accessed: Aug. 13, 2026. [Online]. Available: <https://github.com/systemd/systemd/commit/b67f541f130cd4c55da0b74af5fcbb4daeca1937>
- [54] L. Poettering, “Commit 4ce534f - journal: use a different hash function for each journal file .” Accessed: Aug. 13, 2026. [Online]. Available: <https://github.com/systemd/systemd/commit/4ce534f4cd950cb22abacdfd4d967b41b890a706>
- [55] T. Geng *et al.*, “Prompt Injection Attacks on Large Language Models: A Survey of Attack Methods, Root Causes, and Defense Strategies,” *Computers, Materials and Continua*, vol. 87, no. 1, 2026, doi: <https://doi.org/10.32604/cmc.2025.074081>.
- [56] S. Thornton, “Can Adversarial Code Comments Fool AI Security Reviewers – Large-Scale Empirical Study of Comment-Based Attacks and Defenses Against LLM Code Analysis.” p. , Feb. 2026. doi: [10.48550/arXiv.2602.16741](https://doi.org/10.48550/arXiv.2602.16741).
- [57] Anthropic, “Pricing .” Accessed: Aug. 25, 2026. [Online]. Available: <https://platform.claude.com/docs/en/about-claude/pricing>
- [58] G. Inc., “Gemini 2.5 .” Accessed: Aug. 25, 2026. [Online]. Available: <https://cloud.google.com/gemini-enterprise-agent-platform/generative-ai/pricing>
- [59] N. Jegham *et al.*, “How Hungry is AI? Benchmarking Energy, Water, and Carbon Footprint of LLM Inference,” May 2025, doi: [10.48550/ARXIV.2505.09598](https://doi.org/10.48550/ARXIV.2505.09598).

## Appendices

|   |                                                                                          |      |
|---|------------------------------------------------------------------------------------------|------|
| 1 | Appendix: .....                                                                          | XV   |
| 2 | Extensive Manual Analysis of the Dataset .....                                           | XVI  |
| 3 | All Classifier Configurations Compared with McNemar Test .....                           | XX   |
| 4 | Detailed Report of all Classifications yielded during Training (Threshold<br>0.58) ..... | XXII |
| 5 | Lora Fine-Tuning Parameters .....                                                        | XXIV |
| 6 | Comparison of Primitives included in Trainings .....                                     | XXV  |

## **1 Appendix:**

## 2 Extensive Manual Analysis of the Dataset

As discussed in Section 4.1.3, the dataset used to train and test the model in this thesis is constructed with the assistance of the Claude Sonnet 4.6 LLM. This makes the data prone to mislabeling, which would affect model performance. To weigh the risk posed by this labeling technique, this analysis was conducted.

The following tables provide the results of an extensive manual analysis of 3.5% of all files in the codebase. In total, 35 random files of all three categories (10 self-implemented crypto, 17 use crypto-APIs and 8 unrelated to crypto) are analyzed while respecting their distribution in the dataset. There are two reasons for maintaining stratification in this sample. Firstly, stratification makes it easier to get significant statistical expression about the whole dataset, despite viewing only a small proportion. Secondly, the identification of mislabeled files is equally important for all categories (crypto and non-crypto). If the non-crypto dataset contains a mislabeled crypto file, the model risks to yield more false negatives as crypto files with similar patterns to the mislabeled file tend to be classified as non-crypto.

For every file, the analysis comprised the following steps:

- 1. Prompt an LLM (Gemini 2.5 Pro) to classify the file in any of the three categories
- 2. Manually review the file to understand its purpose
- 3. Search for hints of self-implemented crypto (e.g., comments, function names, math)
- 4. If crypto is suspected, assume the algorithm used and compare with official implementations

The result of this experiment suggests that there is no discrepancy in how the 35 files are labeled in the dataset and in the extensive analysis. While this does not prove a clean dataset, it indicates a very low risk of a mislabeled file. Concretely, when applying a simple binomial calculation with a confidence of 95% (which is numerically close to  $\frac{3}{n}$ ), a maximum error rate of approximately 8.6% can be estimated [39]. In other words, this manual analysis confidently indicates that at most 8.6% of all files in the dataset are mislabeled.

Files that have self-implemented cryptographic functions:

| File                                     | AI Recommendation | Manual Analysis | Key Indicators                                                                                                                                                                                                                                                       |
|------------------------------------------|-------------------|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| BronzeOxide9_math3092pajsecCrypto25519.c | Contains Crypto   | Contains Crypto | <ul style="list-style-type: none"> <li>Refers to the original Curve25519 paper in a comment</li> <li>Author mentions cryptographic security in comments</li> <li>Contains a test function for cryptographic key exchange using only functions in the file</li> </ul> |
| ml_dsa_ntt_vec128.c                      | Contains Crypto   | Contains Crypto | <ul style="list-style-type: none"> <li>Implements the mathematical baseline for the ML-DSA algorithm</li> <li>Implements important aspects of Montgomery multiplication</li> <li>Variables have names related to ML-DSA</li> </ul>                                   |
| SloanPC_ZeroClient_Crypto.c              | Contains Crypto   | Contains Crypto | <ul style="list-style-type: none"> <li>Initiates the Blowfish constants (digits of Pi)</li> </ul>                                                                                                                                                                    |
| wgpsec_CS-Avoid-killing_evil.cpp         | Contains Crypto   | Contains Crypto | <ul style="list-style-type: none"> <li>Implements the RC4 encryption and decryption function</li> <li>Mentions RC4 in its comments</li> </ul>                                                                                                                        |
| hnsvn_bip39wally-core-native_ctaes.c     | Contains Crypto   | Contains Crypto | <ul style="list-style-type: none"> <li>Implements the AES S-Box</li> <li>Implements AES-Encrypt and AES-Decrypt functions</li> <li>Gives detailed information about AES in comments</li> </ul>                                                                       |
| mplpl_unrar_amiga_blake2s.c              | Contains Crypto   | Contains Crypto | <ul style="list-style-type: none"> <li>Implements the Blake2 initialization</li> <li>Implements the core hashing function</li> <li>Gives detailed information about the Blake2 implementation in comments</li> </ul>                                                 |
| sckott_xpdfcopy_Decrypt.c                | Contains Crypto   | Contains Crypto | <ul style="list-style-type: none"> <li>Implements multiple cryptographic algorithms (SHA-256, SHA-512, AES, etc.)</li> <li>Implements decrypt functions</li> <li>Implements cryptographic constants, e.g. AES</li> </ul>                                             |
| BSzili_aros-stuff_blake2s.c              | Contains Crypto   | Contains Crypto | <ul style="list-style-type: none"> <li>Implements the Blake2 initialization</li> <li>Implements the core hashing function</li> <li>Same file as <code>mplpl_unrar_amiga_blake2s.cpp</code></li> </ul>                                                                |
| chacha_enc.c                             | Contains Crypto   | Contains Crypto | <ul style="list-style-type: none"> <li>Mentions “Adapted from the public domain code by D. Bernstein”</li> <li>Implements a ChaCha quarter-round operation</li> </ul>                                                                                                |
| luismartingarcia_alda6suite_md5.cc       | Contains Crypto   | Contains Crypto | <ul style="list-style-type: none"> <li>Contains a comment describing generation of an MD5 digest</li> <li>Implements the MD5 initialization constants</li> <li>Implements MD5 steps</li> </ul>                                                                       |

Files completely unrelated to cryptography:

| File                              | AI Recommendation       | Manual Analysis         | Key Indicators                                                                                                 |
|-----------------------------------|-------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------|
| grate-driver_linux_clkm2701-aud.c | Does not Contain Crypto | Does not Contain Crypto | <ul style="list-style-type: none"> <li>Does not mention any cryptographic keyword (only as subword)</li> </ul> |

| File                                    | AI Recommendation       | Manual Analysis         | Key Indicators                                                                                                                                                                      |
|-----------------------------------------|-------------------------|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                         |                         |                         | <ul style="list-style-type: none"> <li>• Implementation of an Audio driver for Linux</li> </ul>                                                                                     |
| wuzhouhui_unp_1.c                       | Does not Contain Crypto | Does not Contain Crypto | <ul style="list-style-type: none"> <li>• File with only comments and without code</li> </ul>                                                                                        |
| jpostal_FreeBSD-sandbox_g_concat.c      | Does not Contain Crypto | Does not Contain Crypto | <ul style="list-style-type: none"> <li>• Disk utility tool</li> <li>• Does not mention cryptography</li> </ul>                                                                      |
| KDE_konversation_konsol                 | Does not Contain Crypto | Does not Contain Crypto | <ul style="list-style-type: none"> <li>• Part of the KDE Konversation application</li> <li>• Does not mention cryptography</li> </ul>                                               |
| Nikita11042001_SS_LAB_2                 | Does not Contain Crypto | Does not Contain Crypto | <ul style="list-style-type: none"> <li>• No logic instead of <code>while(true){}</code></li> <li>• Contains a comment about executing an infinite loop in the background</li> </ul> |
| mqudsi_textlist_textlist.cpp            | Does not Contain Crypto | Does not Contain Crypto | <ul style="list-style-type: none"> <li>• Implements a utility tool for strings</li> <li>• Does not mention cryptography</li> </ul>                                                  |
| letiangit_802.11ah-ns3-git_wifi-mode.cc | Does not Contain Crypto | Does not Contain Crypto | <ul style="list-style-type: none"> <li>• Implements a WiFi simulation tool</li> <li>• Does not mention cryptography</li> </ul>                                                      |
| just-js_lo_lo.cc                        | Does not Contain Crypto | Does not Contain Crypto | <ul style="list-style-type: none"> <li>• Implements an environment for JavaScript applications</li> <li>• Does not mention cryptography</li> </ul>                                  |

Files related to Crypto without self-implementations:

| File                                                 | AI Recommendation    | Manual Analysis                       | Key Indicators                                                                                                                                                            |
|------------------------------------------------------|----------------------|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| larryli_PuTTY_settings.c                             | Does use Crypto-APIs | Does configure the use of Crypto-APIs | <ul style="list-style-type: none"> <li>• Configures the use of cryptographic APIs</li> <li>• Lists many cryptographic functions used by PuTTY</li> </ul>                  |
| ydb-platform_ydb-cpp-sdk_export.cpp                  | Does use Crypto-APIs | Does configure the use of Crypto-APIs | <ul style="list-style-type: none"> <li>• Configures the use of cryptographic APIs</li> </ul>                                                                              |
| ydb-platform_ydb-cpp-sdk_export.cpp                  | Does use Crypto-APIs | Does use Crypto-APIs                  | <ul style="list-style-type: none"> <li>• Includes crypto libraries</li> <li>• Does not implement mathematical operations that indicate the use of cryptography</li> </ul> |
| alerj78_lucky7coin_init.cpp                          | Does use Crypto-APIs | Does use Crypto-APIs                  | <ul style="list-style-type: none"> <li>• Includes crypto libraries</li> <li>• Implements a tool for managing suspicious crypto coins</li> </ul>                           |
| gamringer_php-pkcs11_pkcs11.c                        | Does use Crypto-APIs | Does configure the use of Crypto-APIs | <ul style="list-style-type: none"> <li>• Defines many constants relevant for a PHP crypto API</li> <li>• Does not implement much logic</li> </ul>                         |
| Azure_embedded-wireless-framework_snmpv2_framework.c | Does use Crypto-APIs | Does configure the use of Crypto-APIs | <ul style="list-style-type: none"> <li>• Includes a crypto library</li> <li>• Implements little logic</li> </ul>                                                          |
| Fraunhofer-FIT-DIEN_iec104-python_python.cpp         | Does use Crypto-APIs | Does configure the use of Crypto-APIs | <ul style="list-style-type: none"> <li>• Configures the use of TLS</li> <li>• Does not mention any other cryptographic algorithm besides TLS</li> </ul>                   |
| OpenKMIP_libkmip_kmip                                | Does use Crypto-APIs | Does mention the use of Crypto-APIs   | <ul style="list-style-type: none"> <li>• Prints which cryptographic function is used</li> </ul>                                                                           |

| File                                        | AI Recommendation    | Manual Analysis      | Key Indicators                                                                                                                                               |
|---------------------------------------------|----------------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| facebookincubator_velox_EncryptionCryptoApp | Does use Crypto-APIs | Does use Crypto-APIs | <ul style="list-style-type: none"> <li>Includes a crypto library</li> <li>Only uses library calls for cryptography</li> </ul>                                |
| tomaszmrugalski_dibbler_KDEpp               | Does use Crypto-APIs | Does use Crypto-APIs | <ul style="list-style-type: none"> <li>Mentions the use of OpenSSL</li> <li>Crypto keywords only occur in strings and comments</li> </ul>                    |
| lvaccaro_truecrack_Main.c                   | Does use Crypto-APIs | Does use Crypto-APIs | <ul style="list-style-type: none"> <li>Tool that helps crack ciphers (e.g. dictionary attack)</li> <li>uses crypto libraries</li> </ul>                      |
| dkcoinus_dkcoin_src_init.c                  | Does use Crypto-APIs | Does use Crypto-APIs | <ul style="list-style-type: none"> <li>Tool for managing a cryptocurrency</li> <li>Includes OpenSSL</li> </ul>                                               |
| letiangit_802.11ah-ns3-git_wifi-mode.cc     | Does use Crypto-APIs | Does use Crypto-APIs | <ul style="list-style-type: none"> <li>Mentions the use of OpenSSL</li> <li>Crypto keywords only occur in strings and comments</li> </ul>                    |
| BenjaminOlsen_cfx_cfx.c                     | Does use Crypto-APIs | Does use Crypto-APIs | <ul style="list-style-type: none"> <li>Frontend for using crypto APIs</li> <li>Defines commands that can be used to use specific crypto functions</li> </ul> |
| h2o_picotls_cli.c                           | Does use Crypto-APIs | Does use Crypto-APIs | <ul style="list-style-type: none"> <li>Includes OpenSSL</li> <li>Calls OpenSSL to encrypt data</li> </ul>                                                    |
| PN-CAF_android_system_VoluntaryCryptager    | Does use Crypto-APIs | Does use Crypto-APIs | <ul style="list-style-type: none"> <li>Includes OpenSSL</li> <li>Uses the OpenSSL MD5 implementation</li> </ul>                                              |
| weolar_miniblink49_ssl_priv                 | Does use Crypto-APIs | Does use Crypto-APIs | <ul style="list-style-type: none"> <li>Includes OpenSSL</li> <li>No indications of self-implemented cryptography</li> </ul>                                  |



### 3 All Classifier Configurations Compared with McNemar Test

In Section 4.4.2, 4860 different classifier configurations are tested. The following table shows, how the values of a parameter compare with each other. For example: All neural network classifiers with the 3-layered architecture yield an accumulated total of 13916 false positives and 9144 false negatives. Compared with the totals of the 1-layered architecture by applying a McNemar test, this results in a p-value of  $\approx 0.015$ , which is lower than the statistical significance of 0.05, indicating that both groups are different. As all configurations with the 3-layered architecture yield much less errors than configurations using the 1-layered architecture, it can be assumed that 3-layered architectures are the general better choice.

The McNemar test compares the error rates of two classifiers on paired observations, rather than metrics like the F1-score. Unlike the F1-score, which can be affected by class imbalances in the evaluation data, the error rate is not directly affected by class imbalance and therefore represents the overall proportion of misclassified observations. Consequently, the McNemar test shows whether two models differ significantly in their classification errors on the same observations, rather than whether they differ in F1-score.

Pairwise McNemar Test p-values & Error Counts by Hyperparameter

| Global: Base Model                |         |       |           |
|-----------------------------------|---------|-------|-----------|
|                                   | Alibaba | Qwen  | microsoft |
| Alibaba<br>(FP:7100, FN:11928)    | 1.000   | .004  | <.001     |
| Qwen<br>(FP:12016, FN:17124)      | .004    | 1.000 | <.001     |
| microsoft<br>(FP:19604, FN:17196) | <.001   | <.001 | 1.000     |

| Neural Network: Epochs    |       |       |       |       |       |
|---------------------------|-------|-------|-------|-------|-------|
|                           | 50    | 100   | 150   | 200   | 250   |
| 50<br>(FP:5832, FN:5200)  | 1.000 | .824  | .481  | .804  | .238  |
| 100<br>(FP:6184, FN:5036) | .824  | 1.000 | .109  | 1.000 | .302  |
| 150<br>(FP:6256, FN:4096) | .481  | .109  | 1.000 | .344  | .968  |
| 200<br>(FP:6488, FN:3800) | .804  | 1.000 | .344  | 1.000 | .481  |
| 250<br>(FP:6648, FN:3800) | .238  | .302  | .968  | .481  | 1.000 |

| Random Forest: Estimators (Trees) |       |       |       |       |       |
|-----------------------------------|-------|-------|-------|-------|-------|
|                                   | 50    | 100   | 150   | 200   | 250   |
| 50<br>(FP:1988, FN:4712)          | 1.000 | <.001 | <.001 | <.001 | <.001 |
| 100<br>(FP:1244, FN:4924)         | <.001 | 1.000 | .295  | .165  | .228  |
| 150<br>(FP:1304, FN:4936)         | <.001 | .295  | 1.000 | .770  | .817  |
| 200<br>(FP:1340, FN:4920)         | <.001 | .165  | .770  | 1.000 | .968  |
| 250<br>(FP:1436, FN:4824)         | <.001 | .228  | .817  | .968  | 1.000 |

| Global: Classifier Type |  |  |
|  | NN | RF |
| NN (FP:31408, FN:21932) | 1.000 | <.001 |
| RF (FP:7312, FN:24316) | <.001 | 1.000 |

| Neural Network: Learning Rate |       |       |        |       |       |      |       |       |       |
|-------------------------------|-------|-------|--------|-------|-------|------|-------|-------|-------|
|                               | 5e-5  | 1e-4  | 1.5e-4 | 2e-4  | 3e-4  | 4e-4 | 5e-4  | 6e-4  | 7e-4  |
| 5e-5<br>(FP:3240, FN:2716)    | 1.000 | .845  | .454   | .664  | .851  | .523 | .648  | .968  | .968  |
| 1e-4<br>(FP:3484, FN:2572)    | .845  | 1.000 | .791   | .359  | .424  | .210 | .000  | .815  | .581  |
| 1.5e-4<br>(FP:3436, FN:2404)  | .454  | .791  | 1.000  | .092  | .238  | .077 | .000  | .523  | .227  |
| 2e-4<br>(FP:3504, FN:2340)    | .664  | .359  | .092   | 1.000 | .000  | .000 | .238  | .690  | .791  |
| 2.5e-4<br>(FP:3608, FN:2380)  | .851  | .424  | .238   | 1.000 | .000  | .791 | .332  | .804  | 1.000 |
| 3e-4<br>(FP:3592, FN:2332)    | .523  | .210  | .077   | .000  | 1.000 | .092 | .503  | .581  |       |
| 3.5e-4<br>(FP:3572, FN:2360)  | .648  | .000  | .000   | .238  | .332  | .092 | 1.000 | .648  | .344  |
| 4e-4<br>(FP:3532, FN:2388)    | .968  | .815  | .523   | .690  | .804  | .503 | .648  | 1.000 | .000  |
| 4.5e-4<br>(FP:3440, FN:2440)  | .968  | .581  | .227   | .791  | .000  | .581 | .344  | 1.000 | .000  |

| Random Forest: Max Depth  |       |       |       |       |       |       |
|---------------------------|-------|-------|-------|-------|-------|-------|
|                           | 3     | 6     | 9     | 12    | 15    | None  |
| 3<br>(FP:2376, FN:6480)   | 1.000 | <.001 | <.001 | <.001 | <.001 | <.001 |
| 6<br>(FP:1336, FN:4336)   | <.001 | 1.000 | <.001 | <.001 | <.001 | <.001 |
| 9<br>(FP:1096, FN:3264)   | <.001 | <.001 | 1.000 | .033  | .056  | .620  |
| 12<br>(FP:760, FN:3444)   | <.001 | <.001 | .033  | 1.000 | .901  | .099  |
| 15<br>(FP:824, FN:3392)   | <.001 | <.001 | .056  | .901  | 1.000 | .118  |
| None<br>(FP:920, FN:3400) | <.001 | <.001 | .620  | .099  | .118  | 1.000 |

| Global: Chunk Overlap |  |  |
|  | 0.125 | 0.5 |
| 0.125 (FP:16924, FN:23748) | 1.000 | <.001 |
| 0.5 (FP:21796, FN:22500) | <.001 | 1.000 |

| Neural Network: Batch Size |       |       |       |       |       |
|----------------------------|-------|-------|-------|-------|-------|
|                            | 4     | 8     | 16    | 32    | 64    |
| 4<br>(FP:6852, FN:4268)    | 1.000 | .664  | .146  | 1.000 | 1.000 |
| 8<br>(FP:6520, FN:4304)    | .664  | 1.000 | .648  | .815  | .815  |
| 16<br>(FP:6200, FN:4284)   | .146  | .648  | 1.000 | .332  | .383  |
| 32<br>(FP:6076, FN:4428)   | 1.000 | .815  | .332  | 1.000 | .968  |
| 64<br>(FP:5760, FN:4648)   | 1.000 | .815  | .383  | .968  | 1.000 |

| Random Forest: Min Samples Split |       |       |       |       |       |       |
|----------------------------------|-------|-------|-------|-------|-------|-------|
|                                  | 2     | 3     | 4     | 5     | 6     | 7     |
| 2<br>(FP:1212, FN:4056)          | 1.000 | .380  | .764  | .356  | .890  | .646  |
| 3<br>(FP:1240, FN:4080)          | .380  | 1.000 | .215  | .072  | .556  | .792  |
| 4<br>(FP:1172, FN:4076)          | .764  | .215  | 1.000 | .575  | .636  | .440  |
| 5<br>(FP:1220, FN:3992)          | .356  | .072  | .575  | 1.000 | .280  | .145  |
| 6<br>(FP:1276, FN:4004)          | .890  | .556  | .636  | .280  | 1.000 | .770  |
| 7<br>(FP:1192, FN:4108)          | .646  | .792  | .440  | .145  | .770  | 1.000 |

| Global: Architecture |  |  |
|  | 1 | 3 |
| 1 (FP:17492, FN:12788) | 1.000 | .015 |
| 3 (FP:13916, FN:9144) | .015 | 1.000 |

| Random Forest: Bootstrap     |       |       |
|------------------------------|-------|-------|
|                              | False | True  |
| False<br>(FP:3416, FN:12032) | 1.000 | <.001 |
| True<br>(FP:3896, FN:12284)  | <.001 | 1.000 |

Figure 15: Comparison of all tested classifier configurations using McNemar test on the classifiers error rates. Pairs that can be assumed to be different with a statistical significance of 0.05 are marked green ( $p < 0.05$ ).

## 4 Detailed Report of all Classifications yielded during Training (Threshold 0.58)

Classifier Configuration:

- Alibaba-NLP LLM
- Overlap of 512 Tokens
- 200 estimators
- maximum tree depth of 10
- minimum split of 5
- no bootstrapping
- trained and validated on 40 different combinations of training and test datasets

| Algorithm         | Classi-<br>fied | TP  | FN | Recall  |
|-------------------|-----------------|-----|----|---------|
| 3DES              | 176             | 169 | 7  | 96.02%  |
| AES-GCM           | 95              | 89  | 6  | 93.68%  |
| BLAKE2            | 134             | 44  | 0  | 100.00% |
| BLAKE2s           | 305             | 264 | 0  | 100.00% |
| Bignum            | 85              | 85  | 0  | 100.00% |
| Blowfish          | 189             | 187 | 0  | 100.00% |
| ChaCha20-Poly1305 | 48              | 4   | 0  | 100.00% |
| DES               | 52              | 4   | 1  | 80.00%  |
| ECDH              | 53              | 0   | 2  | 0.00%   |
| ECDSA             | 96              | 0   | 4  | 0.00%   |
| Ed25519           | 48              | 5   | 43 | 10.42%  |
| HC-256            | 2               | 2   | 0  | 100.00% |
| HMAC-SHA256       | 95              | 95  | 0  | 100.00% |
| MD5               | 346             | 261 | 0  | 100.00% |
| Poly1305          | 324             | 318 | 0  | 100.00% |
| RC2               | 41              | 41  | 0  | 100.00% |

| Algorithm | Classi-<br>fied | TP  | FN | Recall  |
|-----------|-----------------|-----|----|---------|
| RC4       | 180             | 102 | 78 | 56.67%  |
| RIPEMD    | 41              | 41  | 0  | 100.00% |
| RSA       | 91              | 1   | 45 | 2.17%   |
| SHA-1     | 72              | 72  | 0  | 100.00% |
| SHA-2     | 44              | 3   | 0  | 100.00% |
| SHA-256   | 129             | 126 | 3  | 97.67%  |
| SHA-512   | 102             | 100 | 2  | 98.04%  |
| Scrypt    | 3               | 3   | 0  | 100.00% |
| Twofish   | 46              | 41  | 0  | 100.00% |
| ZPAQ      | 3               | 3   | 0  | 100.00% |
| x25519    | 90              | 49  | 0  | 100.00% |

## 5 Lora Fine-Tuning Parameters

The Large Language Model “Alibaba-NLP/gte-Qwen2-1.5B-instruct” is fine-tuned to better distinguish crypto code from other code using the QLoRA method. For this, the following parameters are used:

```
1 peft_config = LoraConfig(  
2     r=16,  
3     lora_alpha=32,  
4     target_modules=["q_proj", "v_proj", "k_proj", "o_proj", "gate_proj",  
5         "up_proj", "down_proj"],  
6     lora_dropout=0.05,  
7     bias="none",  
8     task_type=None  
9 )
```

```
1 quantization_config = BitsAndBytesConfig(  
2     load_in_4bit=True,  
3     bnb_4bit_use_double_quant=True,  
4     bnb_4bit_quant_type="nf4",  
5     bnb_4bit_compute_dtype=torch.bfloat16  
6 )
```

## **6 Comparison of Primitives included in Trainings**

|                                 |          |       |          |           |            |
|---------------------------------|----------|-------|----------|-----------|------------|
| Primitive used to Train FoC     | 3des     | aes   | aria     | blake2    | blowfish   |
| Used in Training in this Thesis | ✓        | ✓     | x        | ✓         | ✓          |
| Primitive used to Train FoC     | camellia | cast  | chacha20 | cmac      | curve25519 |
| Used in Training in this Thesis | ✓        | x     | ✓        | x         | x          |
| Primitive used to Train FoC     | curve448 | des   | dh       | dsa       | ecc        |
| Used in Training in this Thesis | x        | ✓     | x        | x         | x          |
| Primitive used to Train FoC     | ecdh     | ecdsa | ecjpake  | ed448     | ed25519    |
| Used in Training in this Thesis | ✓        | ✓     | x        | x         | ✓          |
| Primitive used to Train FoC     | hmac     | idea  | md4      | md5       | mdc2       |
| Used in Training in this Thesis | ✓        | x     | x        | ✓         | x          |
| Primitive used to Train FoC     | poly1305 | rc2   | rc4      | ripemd160 | rsa        |
| Used in Training in this Thesis | ✓        | x     | ✓        | x         | ✓          |
| Primitive used to Train FoC     | salsa20  | sha1  | sha224   | sha256    | sha384     |
| Used in Training in this Thesis | x        | ✓     | x        | ✓         | x          |
| Primitive used to Train FoC     | sha512   | sha3  | siphash  | sm2       | sm3        |
| Used in Training in this Thesis | ✓        | x     | x        | x         | x          |
| Primitive used to Train FoC     | sm4      | tea   | umac     | whirlpool | xtea       |
| Used in Training in this Thesis | x        | x     | x        | x         | x          |

this Thesis

Table 12: This table shows all primitives that are included in the training of the FoC tool [35]. Additionally, it shows whether the primitive can be found in the training dataset of this thesis' detection model.